CHAPTER XVI

Self-Ref and Self-Rep

IN THIS CHAPTER, we will look at some of the mechanisms which create self-reference in various contexts, and compare them to the mechanisms which allow some kinds of systems to reproduce themselves. Some remarkable and beautiful parallels between these mechanisms will come to light.

Implicitly and Explicitly Self-Referential Sentences

To begin with, let us look at sentences which, at first glance, may seem to provide the simplest examples of self-reference. Some such sentences are these:

(1) This sentence contains five words.
(2) This sentence is meaningless because it is self-referential.
(3) This sentence no verb.
(4) This sentence is false. (Epimenides paradox)
(5) The sentence I am now writing is the sentence you are now reading.

All but the last one (which is an anomaly) involve the simple-seeming mechanism contained in the phrase "this sentence". But that mechanism is in reality far from simple. All of these sentences are "floating" in the context of the English language. They can be compared to icebergs, whose tips only are visible. The word sequences are the tips of the icebergs, and the processing which must be done to understand them is the hidden part. In this sense their meaning is implicit, not explicit. Of course, no sentence's meaning is completely explicit, but the more explicit the self-reference is, the more exposed will be the mechanisms underlying it. In this case, for the self-reference of the sentences above to be recognized, not only has one to be comfortable with a language such as English which can deal with linguistic subject matter, but also one has to be able to figure out the referent of the phrase "this sentence". It seems simple, but it depends on our very complex yet totally assimilated ability to handle English. What is especially important here is the ability to figure out the referent of a noun phrase with a demonstrative adjective in it. This ability is built up slowly, and should by no means be considered trivial. The difficulty is perhaps underlined when a sentence such as number 4 is presented to someone naive about paradoxes and linguistic tricks, such as a child. They may say, "What sentence is false and it may take a bit of persistence to get across the idea that the sentence is talking about itself. The whole idea is a little mind

boggling at first. A couple of pictures may help (Figs. 83, 84). Figure 83 is a picture which can be interpreted on two levels. On one level, it is a sentence pointing at itself; on the other level, it is a picture of Epimenides executing his own death sentence.

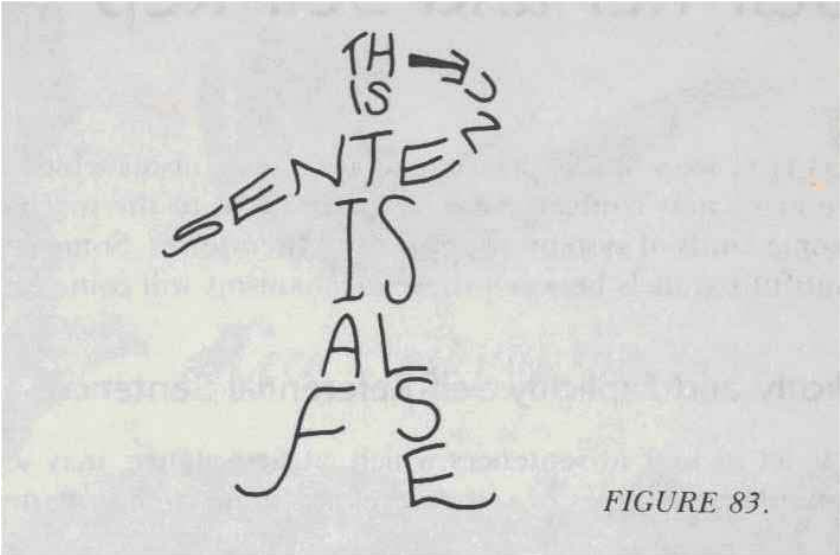


FIGURE 83.

.

Figure 84, showing visible and invisible portions of the iceberg, suggests the relative proportion of sentence to processing required for the recognition of self-reference:

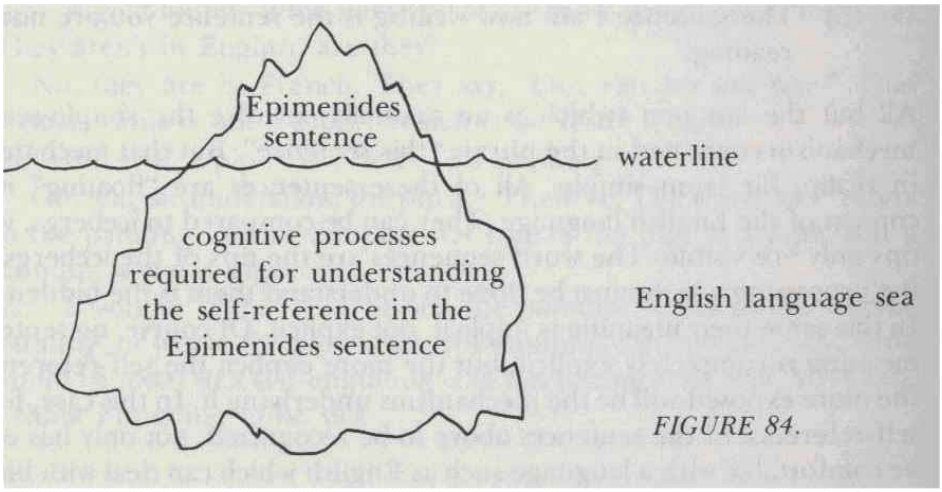


FIGURE 84.

It is amusing to try to create a self-referring sentence without using the trick of saving this sentence". One could try to quote a sentence inside itself. Here is an attempt:

The sentence "The sentence contains five words" contains five words.

But such an attempt must fail, for any sentence that could be quoted entirely inside itself would have to be shorter than itself. This is actually possible, but only if you are willing to entertain infinitely long sentences, such as:

The sentence
"The sentence
"The sentence
"The sentence
...
....
etc,.,etc.
...
....
is infinitely long"
is infinitely long"
is infinitely long'
is infinitely long.

But this cannot work for finite sentences. For the same reason, Godel's string G could not contain the explicit numeral for its Godel number: it would not fit. No string of TNT can contain the TNT-numeral for its own Godel number, for that numeral always contains more symbols than the string itself does. But you can get around this by having G contain a *description* of its own Godel number, by means of the notions of "sub" and "arithmoquinification".

One way of achieving self-reference in an English sentence by means of description instead of by self-quoting or using the phrase "this sentence" is the Quine method, illustrated in the dialogue *Air on G's String*. The understanding of the Quine sentence requires less subtle mental processing than the four examples cited earlier. Although it may appear at first to be trickier, it is in some ways more explicit. The Quine construction is quite like the Godel construction, in the way that it creates self-reference by describing another typographical entity which, as it turns out, is isomorphic to the Quine sentence itself. The description of the new typographical entity is carried out by two parts of the Quine sentence. One part is a set of instructions telling how to build a certain phrase, while the other part contains the construction materials to be used; that is, the other part is a *template*. This resembles a floating cake of soap more than it resembles an iceberg (See Fig. 85).

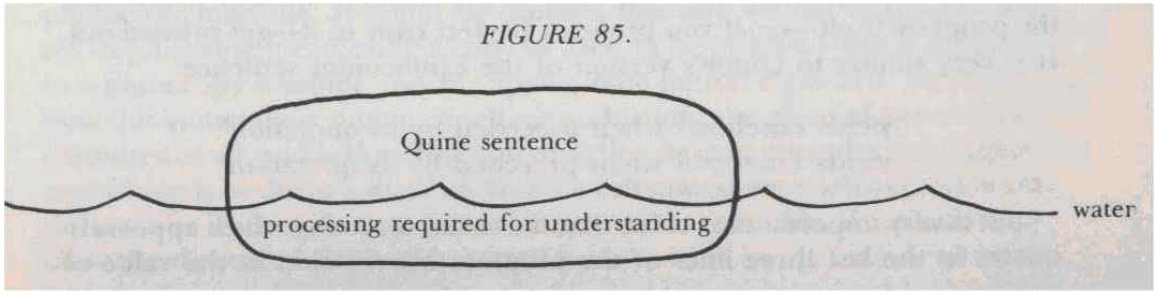


FIGURE 85.

The self-reference of this sentence is achieved in a more direct way than in the Epimenides paradox; less hidden processing is needed. By the way, it is interesting to point out that the phrase "this sentence" appears in the previous sentence; yet it is not there to cause self-reference: you probably understood that its referent was the Quine sentence, rather than the sentence in which it occurs. This just goes to show how pointer phrases such as "this sentence" are interpreted according to context, and helps to show that the processing of such phrases is indeed quite involved.

A Self-Reproducing Program

The notion of quining, and its usage in creating self-reference, have already been explained inside the Dialogue itself, so we need not dwell on such matters here. Let us instead show how a computer program can use precisely the same technique to reproduce itself. The following selfreproducing program is written in a BlooP-like language and is based on following a phrase by its own quotation (the opposite order from quining, so I reverse the name "quine" to make "eniuq"):

```
DEFINE PROCEDURE "ENIUQ" [TEMPLATE]: PRINT [TEMPLATE, LEFT-
BRACKET, QUOTE-MARK, TEMPLATE, QUOTE-MARK, RIGHT-BRACKET,
PERIOD].

ENIUQ
   ['DEFINE PROCEDURE "ENIUQ" [TEMPLATE]: PRINT [TEMPLATE, LEFT-
   BRACKET, QUOTE-MARK, TEMPLATE, QUOTE-MARK, RIGHT-BRACKET,
   PERIOD]. ENIUQ'].
```

ENIUQ is a procedure defined in the first two lines, and its input is called "**TEMPLATE**". It is understood that when the procedure is called, **TEMPLATE's** value will be some string of typographical characters. The effect of **ENIUQ** is to carry out a printing operation, in which TEMPLATE gets printed twice: the first time just plain; the second time wrapped in (single) quotes and brackets, and garnished with a final period. Thus, if **TEMPLATE's** value were the string **DOUBLE-BUBBLE**, then performing **ENIUQ** on it would yield:

DOUBLE-BUBBLE ['DOUBLE-BUBBLE'].

Now in the last four lines of the program above, the procedure **ENIUQ** is called with a specific value of **TEMPLATE**-namely the long string inside the single quotes: **DEFINE** ... **ENIUQ**. That value has been carefully chosen; it consists of the definition of **ENIUQ**, followed by the word **ENIUQ**. This makes the program itself-or, if you prefer, a perfect copy of it-get printed out. It is very similar to Quine's version of the Epimenides sentence:

"yields falsehood when preceded by its quotation"
yields falsehood when preceded by its quotation.

It is very important to realize that the character string which appears n quotes in the last three lines of the program above-that is, the value of

TEMPLATE-is never interpreted as a sequence of instructions. That it happens to be one is, in a sense. just an accident. As was pointed out above, it could just as well have been **DOUBLE-BUBBLE** or any other string of characters. The beauty of the scheme is that when the same string appears in the top two lines of this program, it is treated as a program (because it is not in quotes). Thus in this program, one string functions in two ways: first as program, and second as data. This is the secret of self-reproducing programs, and, as we shall see, of self-reproducing molecules. It is useful, incidentally, to call any kind of self-•reproducing object or entity a *self-rep*; and likewise to call any self-referring object or entity a *self-ref*. I will use those terms occasionally from here on.

The preceding program is an elegant example of a self-reproducing program written in a language which was not designed to make the writing of self-reps particularly easy. Thus, the task had to be carried out using those notions and operations which were assumed to be part of the language-such as the word **QUOTE-MARK**, and the command **PRINT**. But suppose a language were designed expressly for making self-reps easy to write. Then one could write much shorter self-reps. For example, suppose that the operation of eniuq-ing were a built-in feature of the language, needing no explicit definition (as we assumed PRINT was). Then a teeny self-rep would be this:

ENIUQ ['ENIUQ'].

It is very similar to the Tortoise's version of Quine's version of the Epimenides self-ref, where the verb "to quine" is assumed to be known:

"yields falsehood when quined" yields falsehood when quined

But self-reps can be even shorter. For instance, in some computer language it might be a convention that any program whose first symbol is an asterisk is to be copied before being executed normally. Then the program consisting of merely one asterisk is a self-rep! You may complain that this is silly and depends on a totally arbitrary convention. In doing so, you are echoing my earlier point that it is almost cheating to use the phrase "this sentence" to achieve self-reference-it relies too much on the processor, and not enough on explicit directions for self-reference. Using an asterisk as an example of a self-rep is like using the word "I" as an example of a self-ref: both conceal all the interesting aspects of their respective problems.

This is reminiscent of another curious type of self-reproduction: via photocopy machine. It might be claimed that any written document is a self-rep because it can cause a copy of itself to be printed when it is placed in a photocopy machine and the appropriate button is pushed. But somehow this violates our notion of self-reproduction; the piece of paper is not consulted at all, and is therefore not directing its own reproduction. Again, everything is in the processor. Before we call something a self-rep, we want to have the feeling that, to the maximum extent possible, it explicitly contains the directions for copying itself.

To be sure, explicitness is a matter of degree; nonetheless there is an intuitive borderline on one side of which we perceive true self-directed self-reproduction, and on the other side of which we merely see copying being carried out by an inflexible and autonomous copying machine.

What Is a Copy?

Now in any discussion of self-refs and self-reps, one must sooner or later come to grips with the essential issue: what is a copy? We already dealt with that question quite seriously in Chapters V and VI; and now we come back to it. To give the flavor of the issue, let us describe some highly fanciful, yet plausible, examples of self-reps.

A Self-Reproducing Song

Imagine that there is a nickelodeon in the local bar which, if you press buttons 11-U, will play a song whose lyrics go this way:

Put another nickel in, in the nickelodeon,
All I want is 11-U, and music, music, music.

We could make a little diagram of what happens one evening (Fig. 86).

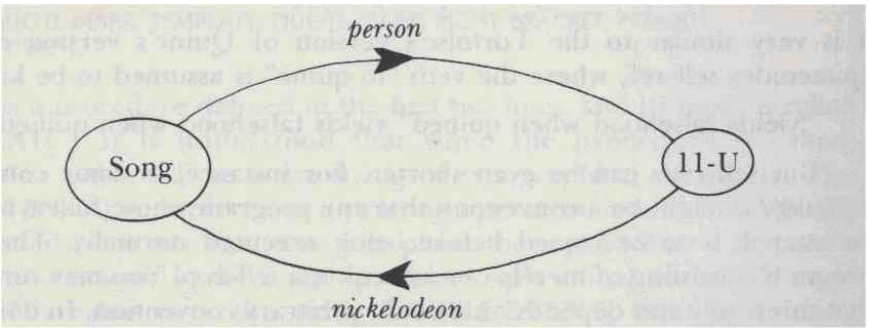


FIGURE 86. A self-reproducing song.

Although the effect is that the song reproduces itself, it would feel strange to call the song a self-rep, because of the fact that when it passes through the 11-U stage, not all of the information is there. The information only gets put back by virtue of the fact that it is fully stored in the nickelodeon that is, in one of the arrows in the diagram, not in one of the ovals. It is questionable whether this song contains a complete description of how to get itself played again, because the symbol pair "1 1-U" is only a trigger, not a copy.

A "Crab" Program

Consider next a computer program which prints itself out backwards. (Some readers might enjoy thinking about how to write such a program in

the blooP-like language above, using the given sell-rep as a inouel.) vvouiu this funny program count as a self-rep. Yes, in a way, because a trivial transformation performed on its output will restore the original program. It seems fair to say that the output contains the same information as the program itself, just recast in a simple way. Yet it is clear that someone might look at the output and not recognize it as a program printed backwards. To recall terminology from Chapter VI, we could say that the "inner messages" of the output and the program itself are the same, but they have different "outer messages"-that -is, they must be read by using different decoding mechanisms. Now if one counts the outer message as part of the information-which seems quite reasonable-then the total information is not the same after all, so the program can't be counted as a self-rep.

However, this is a disquieting conclusion, because we are accustomed to considering something and its mirror image as containing the same information. But recall that in Chapter VI, we made the concept of "intrinsic meaning" dependent on a hypothesized universal notion of intelligence. The idea was that, in determining the intrinsic meaning of an object, we could disregard some types of outer message-those which would be universally understood. That is, if the decoding mechanism seems fundamental enough, in some still ill-defined sense, then the inner message which it lets be revealed is the only meaning that counts. In this example, it seems reasonably safe to guess that a "standard intelligence" would consider two mirror images to contain the same information as each other; that is, it would consider the isomorphism between the two to be so trivial as to be ignorable. And thus our intuition that the program is in some sense a fair self-rep, is allowed to stand.

Epimenides Straddles the Channel

Now another far-fetched example of a self-rep would be a program which prints itself our, but translated into a different computer language. One might liken this to the following curious version of the Quine version of the Epimenides self-ref:

> 'lest une expression qui, quand elle est precedee de sa traduction, mise entre guillemets, clans la langue provenant de l'autre tote de la Manche. tree une faussete" is an expression which, when it is preceded by its translation, placed in quotation marks, into the language originating on the other side of the Channel, yields a falsehood.

You might try to write down the sentence which is described by this weird concoction. (Hint: It is not itself-or at least it is not if "itself" is taken in a naive sense.) If the notion of "self-rep by retrograde motion" (i.e., a program which writes itself out backwards) is reminiscent of a crab canon, the notion of "self-rep by translation" is no less reminiscent of "a canon which involves a transposition of the theme into another key.

A Program That Prints Out Its Own Godel Number

The idea of printing out a translation instead of an exact copy of the original program may seem pointless. However, if you wanted to write a self-rep program in BlooP or FlooP, you would have to resort to some such device, for in those languages, **OUTPUT** is always a number, rather than a typographical string. Therefore, you would have to make the program print out its own Godel number: a very huge integer whose decimal expansion codes for the program, character by character, by using three digit codons. The program is coming as close as it can to printing itself, within the means available to it: it prints out a copy of itself in another "space", and it is easy to switch back and forth between the space of integers and the space of strings. Thus, the value of **OUTPUT** is not a mere trigger, like "11-12". Instead, all the information of the original program lies "close to the surface" of the output.

Godelian Self-Reference

This comes very close to describing the mechanism of Godel's self-ref G. After all, that string of TNT contains a description not of itself, but of an integer (the arithmoquinification of u). It just so happens that that integer is an exact "image" of the string G, in the space of natural numbers. Thus, G refers to a translation of itself into another space. We still feel comfortable in calling G a self-referential string, because the isomorphism between the two spaces is so tight that we can consider them to be identical.

This isomorphism that mirrors TNT inside the abstract realm of natural numbers can be likened to the quasi-isomorphism that mirrors the real world inside our brains, by means of symbols. The symbols play quasi-isomorphic roles to the objects, and it is thanks to them that we can think. Likewise, the Godel numbers play isomorphic roles to strings, and it is thanks to them that we can find metamathematical meanings in statements about natural numbers. The amazing, nearly magical, thing about G is that it manages to achieve self-reference despite the fact that the language in which it is written, TNT, seems to offer no hope of referring to its own structures, unlike English, in which it is the easiest thing in the world to discuss the English language.

So G is an outstanding example of a self-ref via translation-hardly the most straightforward case. One might also think back to some of the Dialogues, for some of them, too, are self-refs via translation. For instance, take the *Sonata for Unaccompanied Achilles*. In that Dialogue, several references are made to the Bach Sonatas for unaccompanied violin, and the Tortoise's suggestion of imagining harpsichord accompaniments is particularly interesting. After all, if one applies this idea to the Dialogue itself, one invents lines which the Tortoise is saying; but if one assumes that Achilles' part stands alone (as does the violin), then it is quite wrong to attribute any lines at all to the Tortoise. In any case, here again is a self-ref by means of a mapping which maps Dialogues onto pieces by Bach. And this mapping is

left, of course, for the reader to notice. Yet even if the reader does not notice it, the mapping is still there, and the Dialogue is still a self-ref.

A Self-Rep by Augmentation

We have been likening self-reps to canons. What, then, would be a fair analogue to a canon by augmentation? Here is a possibility: consider a program which contains a dummy loop whose only purpose is to slow up the program. A parameter might tell how often to repeat the loop. A self-rep could be made which prints out a copy of itself, but with the parameter changed, so that when that copy is run, it will run at half the speed of its parent program; and its "daughter" will in turn run at half again the speed, and so on . . . None of these programs prints itself out precisely; yet all clearly belong to a single "family".

This is reminiscent of the self-reproduction of living organisms. Clearly, an individual is never identical to either of its parents; why, then, is the act of making young called "self-reproduction'? The answer is that there is a coarse-grained isomorphism between parent and child; it is an isomorphism which preserves the information about species. Thus, what is reproduced is the class, rather than the instance. This is also the case in the recursive picture Gplot, in Chapter V: that is, the mapping between "magnetic butterflies" of various sizes and shapes is coarse-grained; no two are identical, but they all belong to a single "species", and the mapping preserves precisely that fact. In terms of self-replicating programs, this would correspond to a family of programs, all written in "dialects" of a single computer language; each one can write itself out, but slightly modified, so that it comes out in a dialect of its original language.

A Kimian Self-Rep

Perhaps the sneakiest example of a self-rep is the following: instead of writing a legal expression in the compiler language, you type one of the compiler's own error messages. When the compiler looks at your "program", the first thing it does is get confused, because your "program" is ungrammatical; hence the compiler prints out an error message. All you need to do is arrange that the one it prints out will be the one you typed in. This kind of self-rep, suggested to me by Scott Kim, exploits a different level of the system from the one you would normally approach. Although it may seem frivolous, it may have counterparts in complex systems where self-reps vie against each other for survival, as we shall soon discuss.

What Is the Original?

Besides the question "What constitutes a copy?", there is another fundamental philosophical question concerning self-reps. That is the obverse

side of the coin: "What is the original?" This can best be explained by referring to some examples:

(1) a program which, when interpreted by some interpreter running on some computer, prints itself out;

(2) a program which, when interpreted by some interpreter running on some computer. prints itself out along with a complete copy of the interpreter (which, after all, is also a program);

(3) a program which, when interpreted by some interpreter running on some computer, not only prints itself out along with a complete copy of the interpreter, but also directs a mechanical assembly process in which a second computer, identical to the one on which the interpreter and program are running, is put together.

It is clear that in (1), the program is the self-rep. But in (3), is it the program which is the self-rep, or the compound system of program plus interpreter, or the union of program, interpreter, and processor?

Clearly, a self-rep can involve more than just printing itself out. In fact, most of the rest of this Chapter is a discussion of self-reps in which data, program, interpreter, and processor are all extremely intertwined, and in which self-replication involves replicating all of them at once.

Typogenetics

We are now about to broach one of the most fascinating and profound topics of the twentieth century: the study of "the molecular logic of the living state", to borrow Albert Lehninger's richly evocative phrase. And logic it is, too but of 'a sort more complex and beautiful than any a human mind ever imagined. We will come at it from a slightly novel angle: via an artificial solitaire game which I call Typogenetics-short for "Typographical Genetics". In Typogenetics I have tried to capture some ideas of molecular genetics in a typographical system which, on first sight, resembles very much the formal systems exemplified by the **MIU**-system. Of course, Typogenetics involves many simplifications, and therefore is useful primarily for didactic purposes.

I should explain immediately that the field of molecular biology is a field in which phenomena on several levels interact, and that Typogenetics is only trying to illustrate phenomena from one or two levels. In particular, purely chemical aspects have been completely avoided-they belong to a level lower than is here dealt with; similarly, all aspects of classical genetics (viz., nonmolecular genetics) have also been avoided-they belong to a level higher than is here dealt with. I have intended in Typogenetics only to give an intuition for those processes centered on the celebrated *Central Dogma of*

Molecular Biology, enunciated by Francis Crick (one of the co-discoverers of the double-helix structure of DNA):

DNA => RNA => proteins.

It is my hope that with this very skeletal model I have constructed the reader will perceive some simple unifying principles of the field principles which might otherwise be obscured by the enormously intricate interplay of phenomena at many different levels. What is sacrificed is, of course, strict accuracy; what is gained is, I hope, a little insight.

Strands, Bases, Enzymes

The game of Typogenetics involves typographical manipulation on sequences of letters. There are four letters involved:

A C G T.

Arbitrary sequences of them are called *strands*. Thus, some strands are:

GGGG
ATTACCA
CATCATCATCAT

Incidentally, "**STRAND**" spelled backwards begins with "**DNA**". This is appropriate since strands, in Typogenetics, play the role of pieces of DNA (which, in real genetics, are often called "strands"). Not only this, but "**STRAND**" fully spelled out backwards is "**DNA RTS**", which may be taken as an acronym for "**D**NA **R**apid **T**ransit **S**ervice". This, too, is appropriate, for the function of "messenger **RNA**"-which in Typogenetics is represented by strands as well-is quite well characterized by the phrase "**R**apid **T**ransit **S**ervice" for **DNA**, as we shall see later.

I will sometimes refer to the letters **A, C, G, T** as bases, and to the positions which they occupy as units. Thus, in the middle strand, there are seven units, in the fourth of which is found the base **A**.

If you have a strand, you can operate on it and change it in various ways. You can also produce additional strands, either by copying, or by cutting a strand in two. Some operations lengthen strands, some shorten them, and some leave their length alone.

Operations come in packets-that is, several to be performed together, in order. Such a packet of operations is a little like a programmed machine which moves up and down the strand doing things to it. These mobile machines are called "typographical enzymes"-enzymes for short. Enzymes operate on strands one unit at a time, and are said to be "bound" to the unit they are operating on at any given moment.

I will show how some sample enzymes act on particular strings. The first thing to know is that each enzyme likes to start out bound to a particular letter. Thus, there are four kinds of enzyme-those which prefer

A, those which prefer **C**, etc. Given the sequence of operations which an enzyme performs, you can figure out which letter it prefers, but for now I'll just give them without explanation. Here's a sample enzyme, consisting of three operations:

(1) Delete the unit to which the enzyme is bound (and then bind to the next unit to the right).
(2) Move one unit to the right.
(3) Insert a T (to the immediate right of this unit).

This enzyme happens to like to bind to A initially. And here's a sample strand:

ACA

What happens if our enzyme binds to the left **A** and begins acting? Step I deletes the A, so we are left with **CA**-and the enzyme is now bound to the **C**. Step 2 slides the enzyme rightwards, to the **A**, and Step 3 appends a **T** onto the end to form the strand **CAT**. And the enzyme has done its complete duty: it has transformed **ACA** into **CAT**.

What if it had bound itself to the right A of ACA? It would have deleted that A and moved off the end of the strand. Whenever this happens, the enzyme quits (this is a general principle). So the entire effect would just be to lop off one symbol.
Let's see some more examples. Here is another enzyme:

(1) Search for the nearest pyrimidine to the right of this unit.
(2) Go into Copy mode.
(3) Search for the nearest purine to the right of this unit.
(4) Cut the strand here (viz., to the right of the present unit).

Now this contains the terms "pyrimidine" and "purine". They are easy terms. **A** and **G** are called *purines*, and **C** and **T** are called *pyrimidines*. So searching for a pyrimidine merely means searching for the nearest **C** or **T.**

Copy Mode and Double Strands

The other new term is *Copy mode*. Any strand can be "copied" onto another strand, but in a funny way. Instead of copying **A** onto **A,** you copy it onto **T**, and vice versa. And instead of copying **C** onto **C**, you copy it onto **G**, and vice versa. Note that a purine copies onto a pyrimidine, and vice versa. This is called *complementary base pairing*. The complements are shown below

	Complement	
Purinas	**A** <====> **T**	pyrimidines
	G <====> **C**	

You can perhaps remember this molecular pairing scheme by recalling that **A**chilles is paired with the **T**ortoise, and the **C**rab with his **G**enes.

When "copying" a strand, therefore, you don't actually copy it, but you manufacture its *complementary* strand. And this one will be written upside down above the original strand. Let's see this in concrete terms. Let the previous enzyme act on the following strand (and that enzyme also happens to like to start at **A**):

CAAAGAGAATCCTCTTTGAT

There are many places it could start. Let's take the second A, for example. The enzyme binds to it, then executes step 1: Search for the nearest pyrimidine to the right. Well, this means a **C** or a **T.** The first one is a T somewhere near the middle of the strand, so that's where we go. Now step 2: Copy mode. Well, we just put an upside-down **A** above our **T**. But that's not all, for Copy mode *remains in effect* until it is shut off-or until the enzyme is done, whichever comes first. This means that every base which is passed through by the enzyme while Copy mode is on will get a complementary base put above it. Step 3 says to look for a purine to the right of our **T.** That is the **G** two symbols in from the right-hand end. Now as we move up to that **G**, we must "copy"-that is, create a complementary strand. Here's what that gives:
(editor' s note, I can't print upside down ie it is too much hard work so V = A and D = G upside down)

AGGAGAAAG
CAAAGAGAATCCTCTTTGAT

The last step is to cut the strand. This will yield two pieces:

VDDVDVVVJ
CAAAGAGAATCCTCTTTG

and **AT**.

And the instruction packet is done. We are left with a double strand, however. Whenever this happens, we separate the two complementary strands from each other (general principle); so in fact our end product is a set of three strands:

AT, CAAAGAGGA, and CAAAGAGAATCCTCTTTG

Notice that the upside-down strand has been turned right side up, and thereby right and left have been reversed.

Now you have seen most of the typographical operations which can be carried out on strands. There are two other instructions which should be mentioned. One shuts off Copy mode; the other switches the enzyme from a strand to the upside-down strand above it. When this happens, if you keep the paper right side up, then you must switch "left" and "right" in all the instructions. Or better, you can keep the wording and just turn the paper around so the top strand becomes legible. If the "switch" command is

given, but there is no complementary base where the enzyme is bound at that instant, then the enzyme just detaches itself from the strand, and its job is done.

It should be mentioned that when a "cut" instruction is encountered, this pertains to both strands (if there are two): however, "delete" pertains only to the strand on which the enzyme is working. If Copy mode is on, then the "insert" command pertains to both strands- the base itself into the strand the enzyme is working on, and its complement into the other strand.

If Copy mode is *off*, then the "insert" command pertains only to the one strand, so a blank space must he inserted into the complementary strand.

And, whenever Copy mode is on, "move" and "search" commands require that one manufacture complementary bases to all bases which the sliding enzyme touches. Incidentally, Copy mode is always *off* when an enzyme starts to work. If Copy mode is *off*, and the command "Shut off copy mode" is encountered, nothing happens. Likewise, If Copy mode is already *on*, and the command "Turn copy mode on" is encountered, then nothing happens.

Amino Acids

There are fifteen types of command, listed below:

Cut	cut strand(s)
del	delete a base from strand
swi	switch enzyme to other strand
mvr	move one unit to the right
mvl	move one unit to the left
cop	turn on Copy mode
off	turn off Copy mode
ina	insert A to the right of this unit
inc	insert C to the right of this unit
ing	insert G to the right of this unit
int	insert T to the right of this unit
rpy	search for the nearest pyrimidine to the right
rpu	search for the nearest purine to the right
Ipy	search for the nearest pyrimidine to the left
lpu	search for the nearest purine to the left

Each one has a three-letter abbreviation. We shall refer to the three-letter abbreviations of commands as amino acids. Thus, *every enzyme is made up of a sequence of amino acids*. Let us write down an arbitrary enzyme:

rpu - inc - cop - myr - tnyl - swi - Ipu - int

and an arbitrary strand:

TAGATCCAGTCCATCGA

and see how the enzyme acts on the strand. It so happens that the enzyme binds to **G** only. Let us bind to the middle **G** and begin. Search rightwards for a purine (viz., **A** or **G**). We (the enzyme) skip over **TCC** and land on **A.**
Insert a **C**. Now we have

TAGATCCAGTCCACTCGA

where the arrow points to the unit to which the enzyme is bound. Set Copy mode. This puts an upside-down **G** above the **C**. Move right, move left, then switch to the other strand. Here's what we have so far:

7V
TAGATCCAGTCCACTCGA

Let's turn it upside down, 'so that the enzyme is attached to the lower strand:

VDJIDV»1DVDD1VDV1
AG

Now we search leftwards for a purine and find A. Copy mode is on, but the complementary bases are already there, so nothing is added. Finally, we insert a T (in Copy mode), and quit:

VD)IVJVJJl7V)DIVDV1
ATG

Our final product is thus two strands:

ATG, and **TAGATCCAGTCCACATCGA**

The old one is of course gone.

Translation and the Typogenetic Code

Now you might be wondering where the enzymes and strands come from, and how to tell the initial binding-preference of a given enzyme. One way might be just to throw some random strands and some random enzymes together, and see what happens when those enzymes act on those strands and their progeny. This has a similar flavor to the MU-puzzle, where there were some given rules of inference and an axiom, and you just began. The only difference is that here, every time a strand is acted on, its original form is gone forever. In the **MU**-puzzle, acting on **MI** to make **MIU** didn't destroy **MI**

But in Typogenetics, as in real genetics, the scheme is quite a bit trickier. We do begin with some arbitrary strand, somewhat like an axiom in a formal system. But we have, initially, no "rules of inference"-that is, no enzymes. However, we can *translate* each strand into one or more enzymes! Thus, the strands themselves will dictate the operations which will be performed upon them, and those operations will in turn produce

new strands which will dictate further enzymes, etc. etc.! This is mixing levels with a vengeance! Think, for the sake of comparison, how different the **MU**-puzzle would have been if each new *theorem* produced could have been turned into a new *rule of inference* by means of some code.

How is this "translation" done? It involves a *Typogenetic Code* by which adjacent pairs of bases-called "duplets"-in a single strand represent different amino acids. There are sixteen possible duplets: **AA, AC, AG, AT, CA, CC**, etc. And there are fifteen amino acids. The Typogenetic Code is shown in Figure 87.

First Base \ Second Base	A	C	G	T
A		cut *s*	del *s*	swi *r*
C	mvr *s*	mvl *s*	cop *r*	off *l*
G	ina *s*	inc *r*	ing *r*	int *l*
T	rpy *r*	rpu *l*	lpy *l*	lpu *l*

FIGURE 87. The Typogenetic Code, by which each duplet in a strand codes for one of fifteen "amino acids" (or a punctuation mark).

According to the table, the translation of the duplet GC is "inc" ("insert a C"); that of AT is "swi" ("switch strands"); and so on. Therefore it becomes clear that a strand can dictate an enzyme very straightforwardly. For example, the strand

TAGATCCAGTCCACATCGA

breaks up into duplets as follows:

TA GA TC CA GT CC AC AT CG A

with the **A** left over at the end. Its translation into an enzyme is:

rpy - ina - rpu - mvr - int - mvl - cut - swi - cop.

(Note that the leftover A contributes nothing.)

Tertiary Structure of Enzymes

What about the little letters 's', 'l', and 'r' in the lower righthand corner of each box% They are crucial in determining the enzyme's binding-preference, and in a peculiar way. In order to figure out what letter an enzyme likes to bind to, you have to figure out the enzyme's "tertiary structure", which is itself determined by the enzyme's "primary structure". By its

primary structure is meant its amino acid sequence. By its tertiary structure is meant the way it likes to "fold up". The point is that enzymes don't like being in straight lines, as we have so far exhibited them. At each internal amino acid (all but the two ends), there is a possibility of a "kink", which is dictated by the letters in the corners. In particular, '1' and 'r' stand for "left" and "right", and 's' stands for "straight". So let us take our most recent sample enzyme, and let it fold itself up to show its tertiary structure. We will start with the enzyme's primary structure, and move along it from left to right. At each amino acid whose corner-letter is '1' we'll put a left turn, for those with 'r', we'll put a right turn, and at 's' we'll put no turn. In Figure 88 is shown the two-dimensional conformation for our enzyme.

Cop

```
swi <== cut <== mvl   <== int
                          mvr
        rpy -==>  ima  ==> rpu
```

FIGURE 88. The tertiary structure of a typoenzyme.

Note the left-kink at "rpu", the right-kink at "swi", and so on. Notice also that the first segment ("rpy z> ina") and the last segment ("swi => cop") are perpendicular. This is the key to the binding-preference. In fact, the *relative orientation of the first and last segments* of an enzyme's tertiary structure determines the binding-preference of the enzyme. We can always orient the enzyme so that its first segment points to the right. If we do so, then the last segment determines the binding-preference, as shown in Figure 89.

FIGURE 89. Table of binding-preferences for typoenzymes.

First Segment	*Last Segment*	*Binding-letter*
⇒	⇒	A
⇒	⇑	C
⇒	⇓	G
⇒	⇐	T

So in our case, we have an enzyme which likes the letter C. If, in folding up, an enzyme happens to cross itself, that's okay-just think of it as going under or over itself. Notice that all its amino acids play a role in the determination of an enzyme's tertiary structure.

Punctuation, Genes, and Ribosomes

Now one thing remains to he explained. Why is there a blank in box **AA** of the typogenetic Code' The answer is that the duplet **AA** acts as a punctuation mark inside a strand, and it signals the end of the code for an enzyme.
That is to say, one strand may code for two or more enzymes if it has one or more duplets **AA** in it. For example, the strand

CG GA TA CT AA AC CG A

Codes for two enzymes

cop - ina - rpy - off
and
cut – cop

with the **AA** serving to divide the strand up into two "genes". The definition of gene is: that portion of a strand which codes for a single enzyme. Note that the mere presence of **AA** inside a strand does not mean that the strand codes for two enzymes. For instance, **CAAG** codes for "mvr - del". The **AA** begins on an even-numbered unit and therefore is not read as a duplet!

The mechanism which reads strands and produces the enzymes which are coded inside them is called a ribosome. (In Typogenetics, the player of the game does the work of' the ribosomes.) Ribosomes are not in any way responsible for the *tertiary* structure of enzymes, for that is entirely determined once the *primary* structure is created. Incidentally, the process of *translation* always goes *from strands to enzymes*, and never in the reverse direction.

Puzzle: A Typogenetical Self-Rep

Now that the rules of Typogenetics have been fully set out, you may find it interesting to experiment with the game. In particular, it would he most interesting to devise a self-replicating strand. This would mean something along the following lines. A single strand is written down. A ribosome acts on it, to produce any or all of the enzymes which are coded for in the strand. Then those enzymes are brought into contact with the original strand, and allowed to work on it. This yields a set of "daughter strands". The daughter strands themselves pass through the rihosomes, to yield a second generation of enzymes, which act on the daughter strands; and the

cycle goes on and on. This can go on for any number of stages; the hope is that eventually, among the strands which are present at some point, there
will be found two copies of the original strand (one of the copies may be, in fact, the original strand).

The Central Dogma of Typogenetics

Typogenetical processes can be represented in skeletal form in a diagram (Fig. 90).

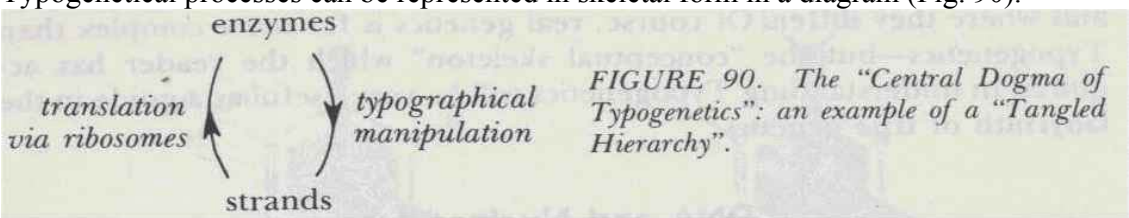
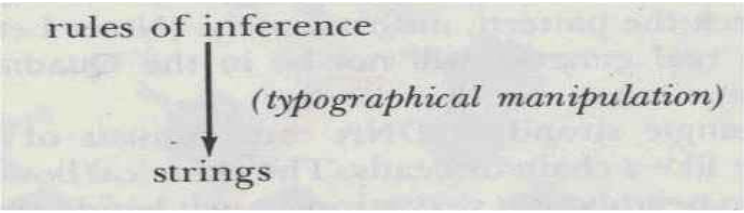


FIGURE 90. The "Central Dogma of Typogenetics": an example of a "Tangled Hierarchy".

This diagram illustrates the *Central Dogma of Typogenetics*. It shows how strands define enzymes (via the Typogenetic Code); and how in turn, enzymes act back on the strands which gave rise to them, yielding new strands. Therefore, the line on the left portrays how old information flows upwards, in the sense that an enzyme is a translation of a strand, and contains therefore the same information as the strand, only in a different form-in particular; in an active form. The line on the right, however, does not show information flowing downwards; instead, it shows how *new information gets created*: by the shunting of symbols in strands.

An enzyme in Typogenetics, like a rule of inference in a formal system, blindly shunts symbols in strands without regard to any "meaning" which may lurk in those symbols. So there is a curious mixture of levels here. On the one hand, strands are acted upon, and therefore play the role of data (as is indicated by the arrow on the right); on the other hand, they also dictate the actions which are to be performed on the data, and therefore they play the role of *programs* (as is indicated by the arrow on the left). It is the player of Typogenetics who acts as interpreter and processor, of course. The two-way street which links "upper" and "lower" levels of Typogenetics shows that, in fact, neither strands nor enzymes can be thought of as being on a higher level than the other. By contrast, a picture of the *Central Dogma of the **MIU***-system looks this way:



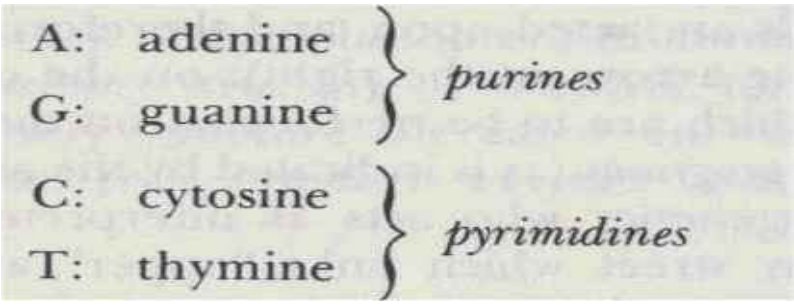
In the **MIU**-system, there is a clear distinction of levels: rules of inference simply belong to a higher level than strings. Similarly for TNT, and all formal systems.

Strange Loops, TNT, and Real Genetics

However, we have seen that in **TNT**, levels are mixed, in another sense. In fact, the distinction between language and metalanguage breaks down: statements about the system get mirrored inside the system. It turns out that if we make a diagram showing the relationship between TNT and its metalanguage, we will produce something which resembles in a remarkable way the diagram which represents the Central Dogma of Molecular Biology. In fact, it is our goal to make this comparison in detail; but to do so, we need to indicate the places where Typogenetics and true genetics coincide, and where they differ. Of course, real genetics is far more complex than Typogenetics-but the "conceptual skeleton" which the reader has acquired in understanding Typogenetics will be very useful as a guide in the labyrinth of true genetics.

DNA and Nucleotides

We begin by discussing the relationship between "strands", and **DNA**. The initials "DNA" stand for "deoxyribonucleic acid". The **DNA** of most cells resides in the cell's nucleus, which is a small area protected by a membrane. Gunther Stent has characterized the nucleus as the "throne room" of the cell, with **DNA** acting as the ruler. DNA consists of long chains of relatively simple molecules called nucleotides. Each nucleotide is made up of three parts: (1) a phosphate group stripped of one special oxygen atom, whence the prefix "deoxy"; (2) a sugar called "ribose", and (3) a base. It is the base alone which distinguishes one nucleotide from another; thus it suffices to specify its base to identify a nucleotide. The four types of bases which occur in **DNA** nucleotides are:



(Also see Fig. 91.) It is easy to remember which ones are pyrimidines because the first vowel in "cytosine", "thymine", and "pyrimidine" is 'y'. Later, when we talk about **RNA**, "uracil"-also a pyrimidine-will come in and wreck the pattern, unfortunately. (Note: Letters representing nucleotides in real genetics will not be in the **Quadrata** font, as they were in Typogenetics.)

A single strand of **DNA** thus consists of many nucleotides strung together like a chain of beads. The chemical bond which links a nucleotide to its two neighbors is very strong; such bonds are called *covalent bonds*, and the "chain of beads" is often called the *covalent backbone* of **DNA**.

Now **DNA** usually comes in double strands-that is, two single strands which are paired up, nucleotide by nucleotide (see Fig. 92). It is the bases

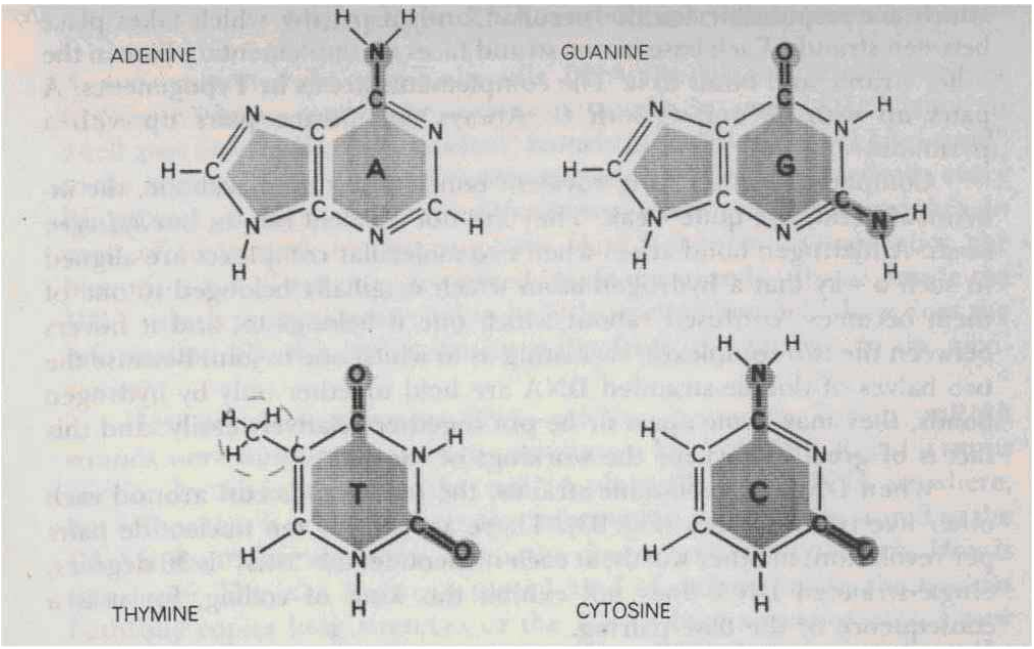
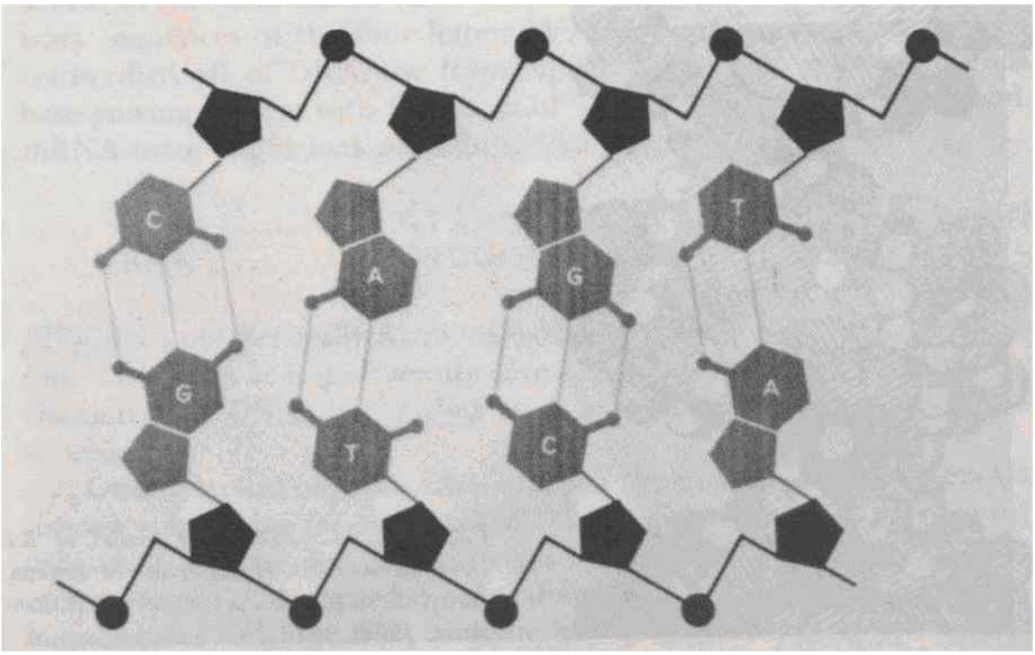


FIGURE 91. The four constituent bases of DNA: Adenine, Guanine, Thymine, Cytosine. [From Hanawalt and Haynes, The Chemical Basis of Life (San Francisco: W. H. Freeman, 1973), p. 142.J

FIGURE 92. DNA structure resembles a ladder in which the side pieces consist of alternating units of deoxyrihose and phosphate. The rungs are formed by the bases paired in a special way, A with T and G with C, and held together respectively by two and three hydrogen bonds. [From Hanawalt and Haynes, The Chemical Basis of Life, p. 142.



which are responsible for the peculiar kind of pairing which takes place between strands. Each base in one strand faces a complementary base in the other strand, and binds to it. The complements are as in Typogenetics: A pairs up with T, and C with G. Always one purine pairs up with a pyrimidine.

Compared to the strong covalent bonds along the backbone, the interstrand bonds are quite weak. They are not covalent bonds, but hydrogen bonds. A hydrogen bond arises when two molecular complexes are aligned in such a way that a hydrogen atom which originally belonged to one of them becomes "confused" about which one it belongs to, and it hovers between the two complexes, vacillating as to which one to join. Because the two halves of double-stranded DNA are held together only by hydrogen bonds, they may come apart or be put together relatively easily; and this fact is of great import for the workings of the cell.

When DNA forms double strands, the two strands curl around each other like twisting vines (Fig. 93). There are exactly ten nucleotide pairs per revolution; in other words, at each nucleotide, the "twist" is 36 degrees. Single-stranded DNA does not exhibit this kind of coiling, for it is a consequence of the base-pairing.

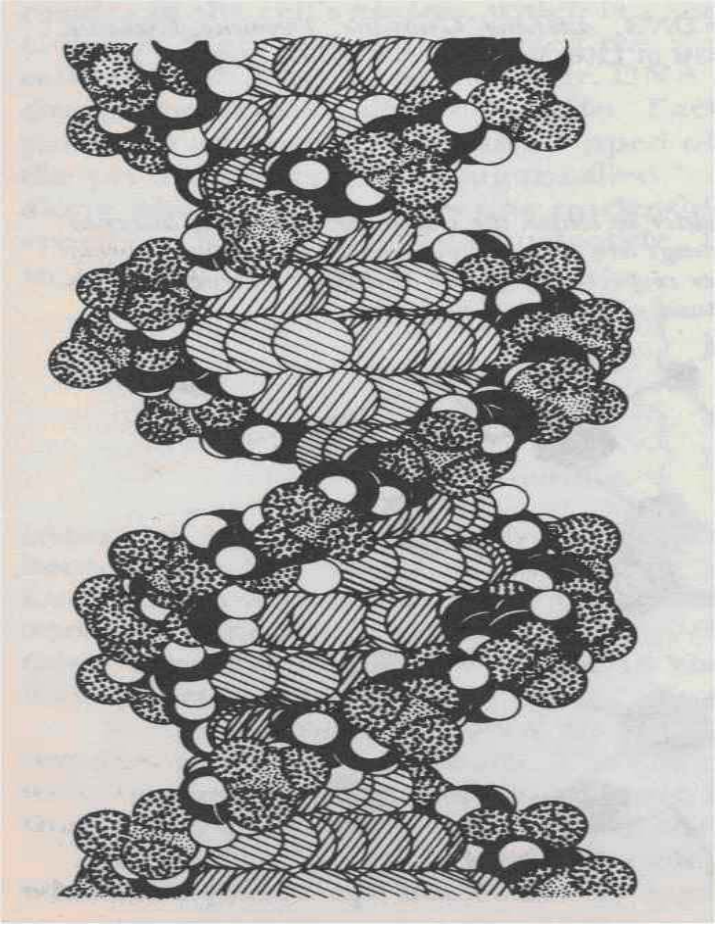


FIGURE 93. Molecular model of the DNA double helix. [From Vernon M. Ingram, Biosynthesis (Menlo Park, Calif.: W. A. Benjamin, 1972)

Messenger KNA and Ribosomes

As was mentioned above, in many cells, DNA, the ruler of the cell, dwells in its private "throne room": the nucleus of the cell. But most of the "living" in a cell goes on outside of the nucleus, namely in the *cytoplasm*-the "ground" to the nucleus' "figure". In particular, *enzymes*, which make practically every life process go, are manufactured by *ribosomes* in the cytoplasm, and they do most of their work in the cytoplasm. And just as in Typogenetics, the blueprints for all enzymes are stored inside the strands-that is, inside the DNA, which remains protected in its little nuclear home. So how does the information about enzyme structure get from the nucleus to the ribosomes'

Here is where *messenger* **RNA**-m**RNA**-comes in. Earlier, m**RNA** strands were humorously said to constitute a kind of **DNA** Rapid Transit Service; by this is meant not that m**RNA** physically carries **DNA** anywhere, but rather that it serves to carry the information, or message, stored in the **DNA** in its nuclear chambers, out to the ribosomes in the cytoplasm. How is this done? The idea is easy: a special kind of enzyme inside the nucleus faithfully copies long stretches of the **DNA's** base sequence onto a new strand-a strand of messenger RNA. This m**RNA** then departs from the nucleus and wanders out into the cytoplasm, where it runs into many ribosomes which begin doing their enzyme-creating work on it.

The process by which **DNA** gets copied onto m**RNA** inside the nucleus is called *transcription*; in it, the double-stranded DNA must be temporarily separated into two single strands, one of which serves as a template for the mRNA. Incidentally, "**RNA**" stands for "ribonucleic acid", and it is very much like **DNA** except that all of its nucleotides possess that special oxygen atom in the phosphate group which **DNA's** nucleotides lack. Therefore the "deoxy" prefix is dropped. Also, instead of thymine, **RNA** uses the' base uracil, so the information in strands of **RNA** can be represented by arbitrary sequences of the four letters **'A', 'C', 'G', 'U'**. Now when mRNA is transcribed off of **DNA**, the transcription process operates via the usual base-pairing (except with **U** instead of **T**), so that a **DNA**-template and its m**RNA**-mate might look something like this:

DNA:	**GGTAAATCAAGTCA**	(template)
m**RNA**:	**GGCAUUUAGUCAGU**	(copy")

RNA does not generally form long double strands with itself, although it can. Therefore it is prevalently found not in the helical form which so characterizes **DNA**, but rather in long, somewhat randomly curving strands.

Once a strand of mRNA has escaped the nucleus, it encounters those strange subcellular creatures called "ribosomes"-but before we go on to explain how a ribosome uses m**RNA**, I want to make some comments about enzymes and proteins. Enzymes belong to the general category of biomolecules called *proteins*, and the job of ribosomes is to make all pro

teins, not just enzymes. Proteins which are not enzymes are much more passive kinds of beings: many of them, for instance, are structural molecules, which means that they are like girders and beams and so forth in buildings: they hold the cell's parts together. There are other kinds of proteins, but for our purposes. the principal proteins are enzymes, and I will henceforth not make a sharp distinction.

Amino Acids

Proteins are composed of sequences of amino acids, which come in twenty primary varieties, each with a three-letter abbreviation:

ala	alanine
arg	arginine
asn	asparagines
asp	aspartic acid
cys	cysteine
gln	glutamine
glu	glutamic acid
gly	glycine
his	histidine
He	isoleucine
leu	leucine
lys	lysine
met	methionine
phe	phenylalanine
pro	praline
ser	serine
thr	threonine
trp	tryptophan
tyr	tyrosine
val	valine

Notice the slight numerical discrepancy with Typogenetics, where we had only fifteen "amino acids" composing enzymes. An amino acid is a small molecule of roughly the same complexity as a nucleotide; hence the building blocks of proteins and of nucleic acids (**DNA**, **RNA**) are roughly of the same size. However, proteins are composed of much shorter sequences of components: typically, about three hundred amino acids make a complete protein, whereas a strand of **DNA** can consist of hundreds of thousands or millions of nucleotides.

Ribosomes and Tape Recorders

Now when a strand of m**RNA**, after its escape into the cytoplasm, encounters a ribosome, a very intricate and beautiful process called translation takes place. It could be said that this process of translation is at the very heart of

all of life, and there are many mysteries connected with it. But in essence it is easy to describe. Let us first give a picturesque image, and then render it more precise. Imagine the mRNA to be like a long piece of magnetic recording tape, and the ribosome to be like a tape recorder. As the tape passes through the playing head of the recorder, it is "read" and converted into music, or other sounds. Thus magnetic markings are "translated" into notes. Similarly, when a "tape" of mRNA passes through the "playing head" of a ribosome, the "notes" which are produced are amine acids, and the "pieces of music" which they make up are proteins. This is what translation is all about: it is shown in Figure 96.

The Genetic Code

But how can a ribosome produce a chain of amino acids when it is reading a chain of nucleotides This mystery was solved in the early 1960's by the efforts of a large number of people, and at the core of the answer lies the Genetic Code-a mapping from triplets of nucleotides into amino acids (see Fig. 94). This is in spirit extremely similar to the Typogenetic Code, except that here, three consecutive bases (or nucleotides) form a codon, whereas there, only two were needed. Thus there must be 4x4x4 (equals 64) different entries in the table, instead of sixteen. A ribosome clicks down a strand of RNA three nucleotides at a time-which is to say, one. codon at a time -and each time it does so, it appends a single new amino acid to the protein it is presently manufacturing. Thus, a protein comes out of the ribosome amino acid by amino acid.

CUA GAU
, C u A g A u

A typical segment of mRNA read first as two triplets (above), and second as three duplets (below): an example of hemiolia in biochemistry

Tertiary Structure

However, as a protein emerges from a ribosome, it is not only getting longer and longer, but it is also continually folding itself up into an extraordinary three-dimensional shape, very much in the way that those funny little Fourth-of-July fireworks called "snakes" simultaneously grow longer and curl up, when they are lit. This fancy shape is called the protein's tertiary structure (Fig. 95), while the amino acid sequence per se is called the primary structure of the protein. The tertiary structure is implicit in the primary structure, just as in Typogenetics. However, the recipe for deriving the tertiary structure, if you know only the primary structure, is by far more complex than that given in Typogenetics. In fact, it is one of the outstanding problems of contemporary molecular biology to figure out some rules by which the tertiary structure of a protein can be predicted if only its primary structure is known.

The Genetic Code.

	U	C	A	G	
U	phe phe leu leu	ser **ser** **ser** ser	tyr tyr *punt.* *punc.*	*cys* *cys* *punt.* trp	U C *A* G
C	leu leu leu Ieu	pro pro pro pro	his his gin gln	arg arg arg arg	U C A G
A	ile ile ile met	thr thr thr thr	asn asn lys lys	**ser** ser arg arg	U C A G
G	val val val val	ala ala ala ala	asp asp glu glu	gly gly gly gly	U C A G

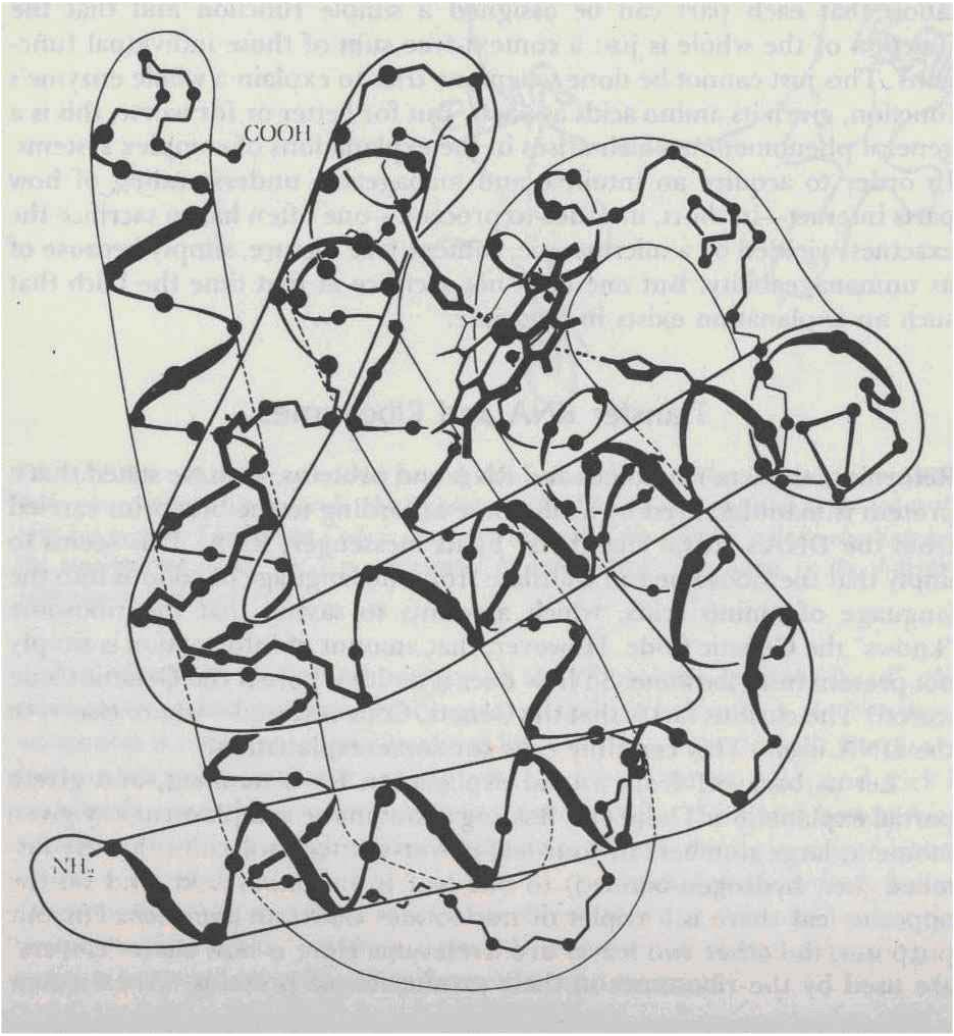
FIGURE 94. The Genetic Code, by which each triplet in a strand of messenger RNA codes for one of twenty amino acids (or a punctuation mark).

Reductionistic Explanation of Protein Function

Another discrepancy between Typogenetics and true genetics-and this is probably the most serious one of all-is this: whereas in Typogenetics, each component amino acid of an enzyme is responsible for some specific "piece of the action", in real enzymes, individual amino acids cannot be assigned such clear roles. It is the tertiary structure as a whole which determines the mode in which an enzyme will function; there is no way one can say, "This

amino acid's presence means that such-and-such an operation will get performed". In other words, in real genetics, an individual amino acid's contribution to the enzyme's overall function is not "context-free". However, this fact should not be construed in any way as ammunition for an anti reductionist argument to the effect that "the whole [enzyme] cannot be explained as the sum of its parts". That would he wholly unjustified. What is justified is rejection of the simpler claim that "each amino acid contributes to the sum in a manner which is independent of the other amino acids present". In other words, the function of a protein cannot be considered to be built up from context-free functions of its parts; rather, one must consider how the parts interact. It is still possible in principle to write a computer program which takes as input the primary structure of a protein,

FIGURE 95. The structure of myoglobin, deduced from high-resolution X-ray data. The large-scale "twisted pipe" appearance is the tertiary structure; the finer helix inside-the "alpha helix"-is the secondary structure. [From A. Lehninger, Biochemistry]



and firstly determines its tertiary structure, and secondly determines the function of the enzyme. This would be a completely reductionistic explanation of the workings of proteins, but the determination of the "sum" of the parts would require a highly complex algorithm. The elucidation of the function of an enzyme, given its primary, or even its tertiary, structure, is another great problem of contemporary molecular biology.

Perhaps, in the last analysis, the function of the whole enzyme can be considered to be built up from functions of parts in a context-free manner, but where the parts are now considered to be individual particles, such as electrons and protons, rather than "chunks", such as amino acids. This exemplifies the "Reductionist's Dilemma": In order to explain everything in terms of context free sums, one has to go down to the level of physics; but then the number of particles is so huge as to make it only a theoretical "in-principle" kind of thing. So, one has to settle for a context-dependent sum, which has two disadvantages. The first is that the parts are much larger units, whose behavior is describable only on a high level, and therefore indeterminately. The second is that the word "sum" carries the connotation that each part can be assigned a simple function and that the function of the whole is just a context-free sum of those individual functions. This just cannot be done when one tries to explain a whole enzyme's function, given its amino acids as parts. But for better or for worse, this is a general phenomenon which arises in the explanations of complex systems. In order to acquire an intuitive and manageable understanding of how parts interact-in short, in order to proceed-one often has to sacrifice the exactness yielded by a microscopic, context-free picture, simply because of its unmanageability. But one does not sacrifice at that time the faith that such an explanation exists in principle.

Transfer RNA and Ribosomes

Returning, then, to ribosomes and RNA and proteins, we have stated that a protein is manufactured by a ribosome according to the blueprint carried from the DNA's "royal chambers" by its messenger, RNA. This seems to imply that the ribosome can translate from the language of codons into the language of amino acids, which amounts to saying that the ribosome "knows" the Genetic Code. However, that amount of information is simply not present in a ribosome. So how does it do it? Where is the Genetic Code stored? The curious fact is that the Genetic Code is stored-where else?-in the DNA itself. This certainly calls for some explanation.

Let us back off from a total explanation for a moment, and give a partial explanation. There are, floating about in the cytoplasm at any given moment, large numbers of four-leaf-clover-shaped molecules; loosely fastened (i.e., hydrogen-bonded) to one leaf is an amino acid, and on the opposite leaf there is a triplet of nucleotides called an anticodon. For our purposes, the other two leaves are irrelevant. Here is how these "clovers" are used by the ribosomes in their production of proteins. When a new

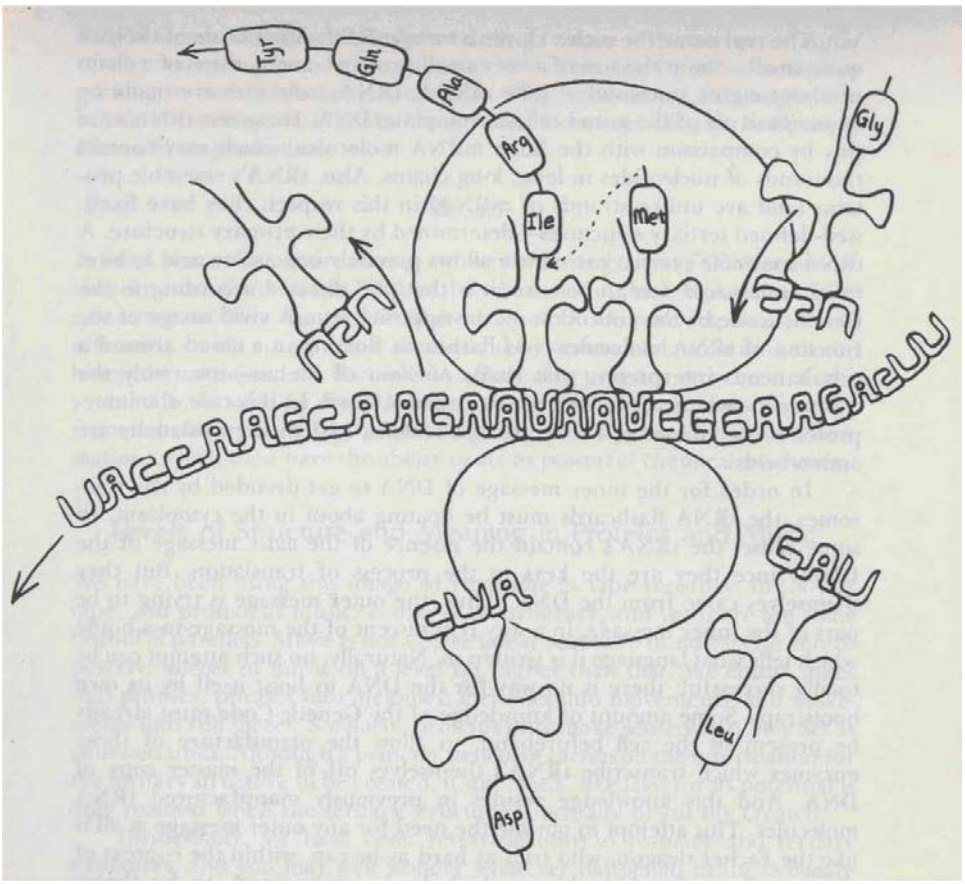


FIGURE 96. A section of mRNA passing through a ribosome. Floating nearby are t**RNA** molecules, carrying amino acids which are stripped off by the ribosome and appended to the growing protein. The Genetic Code is contained in the t**RNA** molecules, collectively. Note how the base-pairing (A-U, C-G) is represented by interlocking letter-forms in the diagram. [Drawing by Scott E. Kim]

codon of m**RNA** clicks into position in the ribosome's "playing head", the ribosome reaches out into the cytoplasm and latches onto a clover whose anticodon is complementary to the m**RNA** codon. Then it pulls the clover into such a position that it can rip off the clover's amino acid, and stick it covalently onto the growing protein. (Incidentally, the bond between an amino acid and its neighbor in a protein is a very strong covalent bond, called a "peptide bond". For this reason, proteins are sometimes called "polypeptides".) Of course it is no accident that the "clovers" carry the proper amino acids, for they have all been manufactured according to precise instructions emanating from the "throne room".

The real name for such a clover is *transfer* **RNA**. A molecule of t**RNA** is quite small-about the size of a very small protein-and consists of a chain of about eighty nucleotides. Like m**RNA**, t**RNA** molecules are made by *transcription* off of the grand cellular template, DNA. However, tRNA's are tiny by comparison with the huge mRNA molecules, which may contain thousands of nucleotides in long, long chains. Also, t**RNA**'s resemble proteins (and are unlike strands of m**RNA**) in this respect: they have fixed, well-defined tertiary structures-determined by their primary structure. A t**RNA** molecule's tertiary structure allows precisely one amino acid to bind to its amino-acid site: to be sure, it is that one dictated according to the Genetic Code by the anticodon on the opposite arm. A vivid image of the function of tRNA molecules is as flashcards floating in a cloud around a simultaneous interpreter, who snaps one out of the air-invariably the right one!-whenever he needs to translate a word. In this case, the interpreter is the ribosome, the words are codons, and their translations are amino acids.

In order for the inner message of **DNA** to get decoded by the ribosomes, the t**RNA** flashcards must be floating about in the cytoplasm. In some sense, the t**RNA**s contain the essence of the outer message of the **DNA**, since they are the keys to the process of translation. But they themselves came from the **DNA**. Thus, the outer message is trying to be part of the inner message, in a way reminiscent of the message-in-a-bottle which tells what language it is written in. Naturally, no such attempt can be totally successful: there is no way for the **DNA** to hoist itself by its own bootstraps. Some amount of knowledge of the Genetic Code must already be present in the cell beforehand, to allow the manufacture of those enzymes which transcribe tRNA's themselves off of the master copy of **DNA**. And this knowledge resides in previously manufactured t**RNA** molecules. This attempt to obviate the need for any outer message at all is like the Escher dragon, who tries as hard as he can, within the context of the two-dimensional world to which he is constrained, to be threedimensional. He seems to go a long way-but of course he never makes it, despite the fine imitation he gives of three-dimensionality.

Punctuation and the Reading Frame

How does a ribosome know when a protein is done? Just as in Typogenetics, there is a signal inside the m**RNA** which indicates the termination or initiation of a protein. In fact, three special codons-**UAA, CAG, UGA** act as *punctuation* marks instead of coding for amino acids. Whenever such a triplet clicks its way into the "reading head" of a ribosome, the ribosome releases the protein under construction and begins a new one.

Recently, the entire genome of the tiniest known virus, φ)X174, has been laid bare. One most unexpected discovery was made en route: some of its nine genes overlap-that is, *two distinct proteins are coded for by the same stretch of **DNA***! There is even one gene contained entirely inside another!

This is accomplished by having the reading frames of the two genes shifted relative to each other, by exactly one unit. The density of information packing in such a scheme is incredible. This is, of course, the inspiration behind the strange "5/17 haiku" in Achilles' fortune cookie, in the *Canon byIntervallic Augmentation*,

Recap

In brief, then, this picture emerges: from its central throne, DNA sends off long strands of messenger RNA to the ribosomes in the cytoplasm; and the ribosomes, making use of the "flashcards" of tRNA hovering about them, efficiently construct proteins, amino acid by amino acid, according to the blueprint contained in the mRNA. Only the primary structure of the proteins is dictated by the DNA; but this is enough, for as they emerge from the ribosomes, the proteins "magically" fold up into complex conformations which then have the ability to act as powerful chemical machines.

Levels of Structure and Meaning in Proteins and Music

We have been using this image of ribosome as tape recorder, mRNA as tape, and protein as music. It may seem arbitrary, and yet there are some beautiful parallels. Music is not a mere linear sequence of notes. Our minds perceive pieces of music on a level far higher than that. We chunk notes into phrases, phrases into melodies, melodies into movements, and movements into full pieces. Similarly, proteins only make sense when they act as chunked units. Although a primary structure carries all the information for the tertiary structure to be created, it still "feels" like less, for its potential is only realized when the tertiary structure is actually physically created.

Incidentally, we have been referring only to primary and tertiary structures, and you may well wonder whatever happened to the secondary structure. Indeed, it exists, as does a quaternary structure, as well. The folding-up of a protein occurs at more than one level. Specifically, at some points along the chain of amino acids, there may be a tendency to form a kind of helix, called the alpha helix (not to be confused with the DNA double helix). This helical twisting of a protein is on a lower level than its tertiary structure. This level of structure is visible in Figure 95. Quaternary structure can be directly compared with the building of 'a musical piece out of independent movements, for it involves the assembly of' several distinct polypeptides, already in their full-blown tertiary beauty, into a larger structure. The binding of these independent chains is usually accomplished by hydrogen bonds, rather than covalent bonds; this is of course just as with pieces of music composed of several movements, which are far less tightly bound to each other than they are internally, but which nevertheless form a tight "organic" whole.

The four levels of primary, secondary, tertiary, and quaternary structure can also be compared to the four levels of the MU-picture (Fig. 60) in

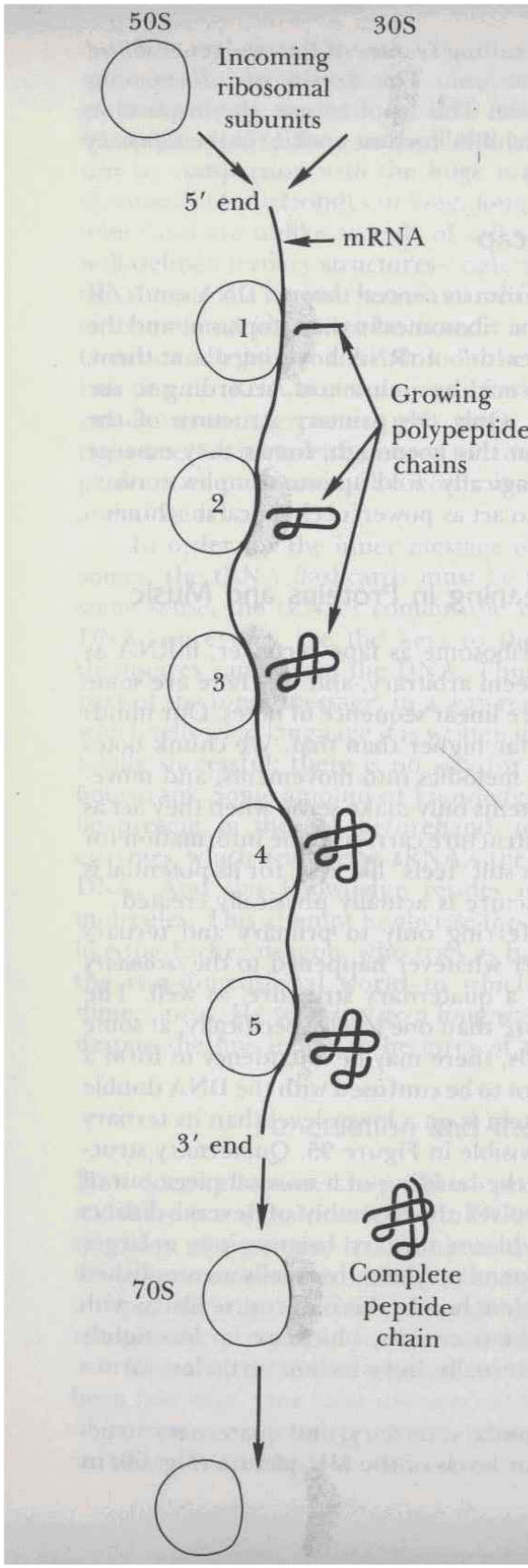


FIGURE 97. A polyribosome. A single strand of mRNA passes through one ribosome after another, like one tape passing through several tape recorders in a row. The result is a set of growing proteins in various stages of completion: the analogue to a musical canon produced by the staggered tape recorders. [*From A. Lehninger,* Biochemistry.]

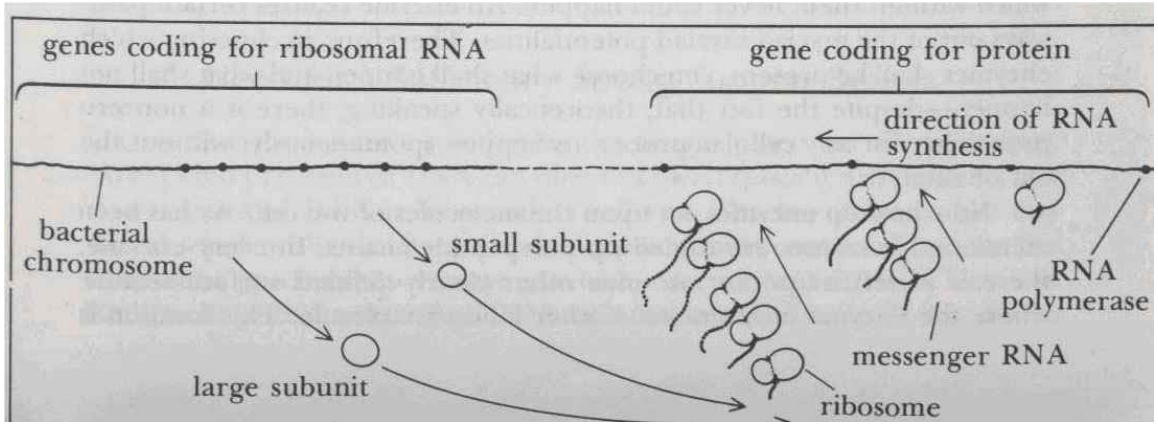
the *Prelude, Ant Fugue*. The global structure-consisting of the letters 'M' and 'U'-is its quaternary structure; then each of those two parts has a tertiary structure, consisting of "HOLISM" or "REDUCTIONISM"; and then the opposite word exists on the secondary level, and at bottom, the primary structure is once again the word "MU", over and over again.

Polyribosomes and Two-Tiered Canons

Now we come to another lovely parallel between tape recorders translating tape into music and ribosomes translating mRNA into proteins. Imagine a collection of many tape recorders, arranged in a row, evenly spaced. We might call this array a "polyrecorder". Now imagine a single tape passing serially through the playing heads of all the component recorders. If the tape contains a single long melody, then the output will be a many-voiced canon, of course, with the delay determined by the time it takes the tape to get from one tape recorder to the next. In cells, such "molecular canons" do indeed exist, where many ribosomes, spaced out in long lines-forming what is called a polyribosome-all "play" the same strand of mRNA, producing identical proteins, staggered in time (see Fig. 97).

Not only this, but nature goes one better. Recall that mRNA is made by transcription off of DNA; the enzymes which are responsible for this process are called RNA polymerases ("-ase" is a general suffix for enzymes). It happens often that a series of RNA polymerases will be at work in parallel on a single strand of DNA, with the result that many separate (but identical) strands of mRNA are being produced, each delayed with respect to the other by the time required for the DNA to slide from one RNA polymerase to the next. At the same time, there can be several different ribosomes working on each of the parallel emerging mRNA's. Thus one arrives at a double-decker, or two-tiered, "molecular canon" (Fig. 98). The corresponding image in music is a rather fanciful but amusing scenario: several

FIGURE 98. Here, an even more complex scheme. Not just one but several strands of mRNA, all emerging by transcription from a single strand of DNA, are acted upon by polyribosomes. The result is a two-tiered molecular canon. [From Hanawalt and Haynes, The Chemical Basis of Life, p. 271]



different copyists are all at work simultaneously, each one of them copying the same original manuscript from a clef which flutists cannot read into a clef which they can read. As each copyist finishes a page of the original manuscript, he passes it on to the next copyist, and starts transcribing a new page himself. Meanwhile, from each score emerging from the pens of the copyists, a set of flutists are reading and tooting the melody, each flutist delayed with respect to the others who are reading from the same sheet. This rather wild image gives, perhaps, an idea of some of the complexity of the processes which are going on in each and every cell of your body during every second of every day ...

Which Came First-The Ribosome or the Protein?

We have been talking about these wonderful beasts called ribosomes; but what are they themselves composed of? How are they made? Ribosomes are composed of two types of things: (1) various kinds of proteins, and (2) another kind of RNA, called ribosomal RNA (rRNA). Thus, in order for a ribosome to be made, certain kinds of proteins must be present, and rRNA must be present. Of course, for proteins to be present, ribosomes must be there to make them. So how do you get around the vicious circle? Which comes first-the ribosome or the protein? Which makes which? Of course there is no answer because one always traces things back to previous members of the same class just as with the chicken-and-the-egg question-until everything vanishes over the horizon of time. In any case, ribosomes are made of two pieces, a large and a small one, each of which contains some rRNA and some proteins. Ribosomes are about the size of large proteins; they are much much smaller than the strands of mRNA which they take as input, and along which they move.

Protein Function

We have spoken somewhat of the structure of proteins-specifically enzymes; but we have not really mentioned the kinds of tasks which they perform in the cell, nor how they do them. All enzymes are catalysts, which means that in a certain sense, they do no more than selectively accelerate various chemical processes in the cell, rather than make things happen which without them never could happen. An enzyme realizes certain pathways out of the myriad myriad potentialities. Therefore, in choosing which enzymes shall be present, you choose what shall happen and what shall not happen-despite the fact that, theoretically speaking, there is a nonzero probability for any cellular process to happen spontaneously, without the aid of catalysts.

Now how do enzymes act upon the molecules of the cell? As has been mentioned, enzymes are folded-up polypeptide chains. In every enzyme, there is a cleft or pocket or some other clearly-defined surface feature where the enzyme hinds to some other kind of molecule. This location is

called its active site, and any molecule which gets bound there is called a substrate. Enzymes may have more than one active site, and more than one substrate. Just as in Typogenetics, enzymes are indeed very choosy about what they will operate upon. The active site usually is quite specific, and allows just one kind of molecule to bind to it, although there are sometimes "decoys"-other molecules which can fit in the active site and clog it up, fooling the enzyme and in fact rendering it inactive.

Once an enzyme and its substrate are bound together, there is some disequilibrium of electric charge, and consequently charge-in the form of electrons and protons-flows around the bound molecules and readjusts itself. By the time equilibrium has been reached, some rather profound chemical changes may have occurred to the substrate. Some examples are these: there may have been a "welding", in which some standard small molecule got tacked onto a nucleotide, amino acid, or other common cellular molecule; a DNA strand may have been "nicked" at a particular location; some piece of a molecule may have gotten lopped off; and so forth. In fact, bio-enzymes do operations on molecules which are quite similar to the typographical operations which Typo-enzymes perform. However, most enzymes perform essentially only a single task, rather than a sequence of tasks. There is one other striking difference between Typoenzymes and bio-enzymes, which is this: whereas Typo-enzymes operate only on strands, bio-enzymes can act on DNA, RNA, other proteins, ribosomes, cell membranes-in short, on anything and everything in the cell. In other words, enzymes are the universal mechanisms for getting things done in the cell. There are enzymes which stick things together and take them apart and modify them and activate them and deactivate them and copy them and repair them and destroy them .. .

Some of the most complex processes in the cell involve "cascades" in which a single molecule of some type triggers the production of a certain kind of enzyme; the manufacturing process begins and the enzymes which come off the "assembly line" open up a new chemical pathway which allows a second kind of enzyme to be produced. This kind of thing can go on for three or four levels, each newly produced type of enzyme triggering the production of another type. In the end a "shower" of copies of the final type of enzyme is produced, and all of the copies go off and do their specialized thing, which may be to chop up some "foreign" DNA, or to help make some amino acid for which the cell is very "thirsty", or whatever.

Need for a Sufficiently Strong Support System

Let us describe nature's solution to the puzzle posed for Typogenetics: "What kind of strand of DNA can direct its own replication?" Certainly not every strand of DNA is inherently a self-rep. The key point is this: any strand which wishes to direct its own copying must contain directions for assembling precisely those enzymes which can carry out the task. Now it is futile to hope that a strand of DNA in isolation could be a self-rep; for in

order for those potential proteins to be pulled out of the DNA, there must not only be ribosomes, but also RNA polymerase, which makes the mRNA that gets transported to the ribosomes. And so we have to begin by assuming a kind of "minimal support system" just sufficiently strong that it allows transcription and translation to be carried out. This minimal support system will thus consist in (1) some proteins, such as RNA polymerase, which allow mRNA to be made from DNA, and (2) some ribosomes.

How DNA Self-Replicates

It is not by any means coincidental that the phrases "sufficiently strong support system" and "sufficiently powerful formal system" sound alike. One is the precondition for a self-rep to arise, the other for a self-ref to arise. In fact there is in essence only one phenomenon going on in two very different guises, and we shall explicitly map this out shortly. But before we do so, let us finish the description of how a strand of DNA can be a self-rep.

The DNA must contain the codes for a set of proteins which will copy it. Now there is a very efficient and elegant way to copy a double-stranded piece of DNA, whose two strands are complementary. This involves two steps:

(1) unravel the two strands from each other;
(2) mate" a new strand to each of the two new single strands.

This process will create two new double strands of DNA, each identical to the original one. Now if our solution is to be based on this idea, it must involve a set of proteins, coded for in the DNA itself, which will carry out these two steps.

It is believed that in cells, these two steps are performed together in a coordinated way, and that they require three principal enzymes: DNA endonuclease, DNA polymerase, and DNA ligase. The first is an "unzipping enzyme": it peels the two original strands apart for a short distance, and then stops. Then the other two enzymes come into the picture. The DNA polymerase is basically a copy-and-move enzyme: it chugs down the short single strands of DNA, copying them complementarily in a fashion reminiscent of the Copy mode in Typogenetics. In order to copy, it draws on raw materials-specifically nucleotides-which are floating about in the cytoplasm. Because the action proceeds in fits and starts, with some unzipping and some copying each time, some short gaps are created, and the DNA ligase is what plugs them up. The process is repeated over and over again. This precision three-enzyme machine proceeds in careful fashion all the way down the length of the DNA molecule, until the whole thing has been peeled apart and simultaneously replicated, so that there are now two copies of it.

Comparison of DNA's Self-Rep Method with Quining

Note that in the enzymatic action on the DNA strands, the fact that information is stored in the DNA is just plain irrelevant; the enzymes are merely carrying out their symbol-shunting functions, just like rules of inference in the MMIU-system. It is of no interest to the three enzymes that at some point they are actually copying the very genes which coded for them. The DNA, to them, is just a template without meaning or interest.

It is quite interesting to compare this with the Quine sentence's method of describing how to construct a copy of itself. There, too, one has a sort of "double strand"-two copies of the same information, where one copy acts as instructions, the other as template. In DNA, the process is vaguely parallel, since the three enzymes (DNA endonuclease, DNA polymerase, DNA ligase) are coded for in just one of the two strands, which therefore acts as program, while the other strand is merely a template. The parallel is not perfect, for when the copying is carried out, both strands are used as template, not just one. Nevertheless, the analogy is highly suggestive. There is a biochemical analogue to the use-mention dichotomy: when DNA is treated as a mere sequence of chemicals to be copied, it is like mention of typographical symbols; when DNA is dictating what operations shall he carried out, it is like use of typographical symbols.

Levels of Meaning of DNA

There are several levels of meaning which can be read from a strand of DNA, depending on how big the chunks are which you look at, and how powerful a decoder you use. On the lowest level, each DNA strand codes for an equivalent RNA strand-the process of decoding being transcription. If one chunks the DNA into triplets, then by using a "genetic decoder", one can read the DNA as a sequence of amino acids. This is translation (on top of transcription). On the next natural level of the hierarchy, DNA is readable as a code for a set of proteins. The physical pulling-out of proteins from genes is called gene expression. Currently, this is the highest level at which we understand what DNA means.

However, there are certain to be higher levels of DNA meaning which are harder to discern. For instance, there is every reason to believe that the DNA of, say, a human being codes for such features as nose shape, music talent, quickness of reflexes, and so on. Could one, in principle, learn to read off such pieces of information directly from a strand of DNA, without going through the actual physical process of epigenesis-the physical pulling-out of phenotype from genotype Presumably, yes, since-in theory-one could have an incredibly powerful computer program simulating the entire process, including every cell, every protein, every tiny feature involved in the replication of DNA, of cells, to the bitter end. The output of such a pseudo-epigenesis program would be a high-level description of the phenotype.

There is another (extremely faint) possibility: that we could learn to read the phenotype off of the genotype without doing an isomorphic simulation of the physical process of epigenesis, but by finding some simpler sort of decoding mechanism. This could be called "shortcut pseudoepigenesis". Whether shortcut or not, pseudo-epigenesis is, of course, totally beyond reach at the present time-with one notable exception: in the species Felis catus, deep probing has revealed that it is indeed possible to read the phenotype directly off of the genotype. The reader will perhaps better appreciate this remarkable fact after directly examining the following typical section of the DNA of Felis catus:

CATCATCATCATCATCATCATCATCATCAT ...

Below is shown a summary of the levels of DNA-readability, together with the names of the different levels of decoding. DNA can be read as a sequence of:

bases (nucleotides)	transcription
amino acids	translation
proteins (primary structure)	. gene expression
proteins (tertiary structure)	
protein clusters	higher levels of gene expression
…	
…. unknown levels of DNA meaning	
….	
(N-1) ????	
(N) physical, mental, and	
psychological traits	pseudo-epigenesis

The Central Dogmap

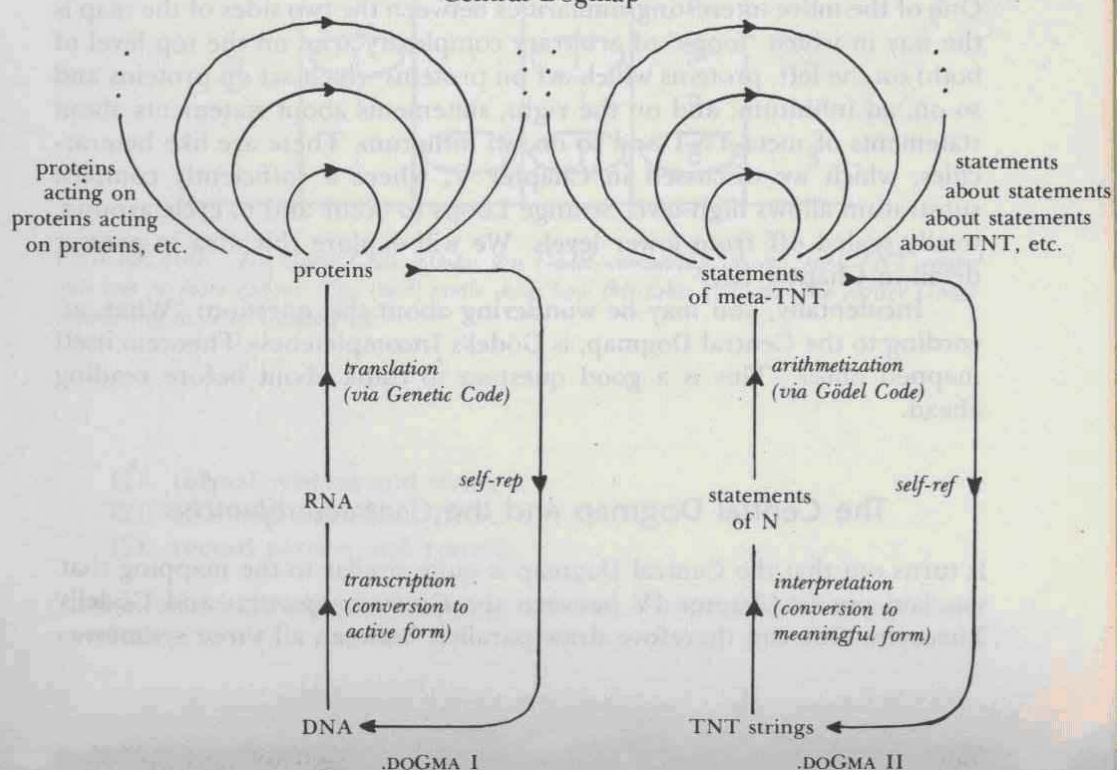
.

With this background, now we are in a position to draw an elaborate comparison between F. Crick's "Central Dogma of Molecular Biology" (.DOGMA I) upon which all cellular processes are based; and what I, with poetic license, call the "Central Dogma of Mathematical Logic" (.DOGMA II), upon which G6del's Theorem is based. The mapping from one onto the other is laid out in Figure 99 and the following chart, which together constitute the Central Dogmap.

FIGURE 99. The Central Dogmap. An analogy is established between two fundamental Tangled Hierarchies: that of molecular biology and that of mathematical logic.

.DOGMA I (Molecular Biology)		.DOGMA II (Mathematical Logic)
strands of DNA	⇔	strings of TNT
strands of mRNA	⇔	statements of N
proteins	⇔	statements of meta-TNT
proteins which act on proteins	⇔	statements about statements of meta-TNT
proteins which act on proteins which act on proteins	⇔	statements about statements about statements of meta-TNT
transcription (DNA ⇒ RNA)	⇔	interpretation (TNT ⇒ N)
Translation (RNA ⇒ proteins)	⇔	Arithmetization (N ⇒ meta-TNT)
Crick	⇔	Gödel
Genetic Code (arbitrary convention)	⇔	Gödel Code (arbitrary convention)
codon (triplet of bases)	⇔	codon (triplet of digits)
amino acid	⇔	quoted symbol of TNT used in meta-TNT
self-reproduction	⇔	self-reference
sufficiently strong cellular support system to permit self-rep	⇔	sufficiently powerful arithmetical formal system to permit self-ref

Central Dogmap



Note the base-pairing of A and T (Arithmetization and Translation), as well as of G and C (Godel and Crick). Mathematical logic gets the purine side, and molecular biology gets the pyrimidine side.

To complete the esthetic side of this mapping, I chose to model my Godel-numbering scheme on the Genetic Code absolutely faithfully. In fact, under the following correspondence, the table of the Genetic Code becomes the table of the Godel Code:

(odd)	1	⇐⇒	A	(purine)
(even)	2	⇐⇒	C	(pyrimidine)
(odd)	3	⇐⇒	G	(purine)
(even)	6	⇐⇒	U	(pyrimidine)

Each amino acid-of which there are twenty-corresponds to exactly one symbol of TNT-of which there are twenty. Thus, at last, my motive for concocting "austere TNT" comes out-so that there would be exactly twenty symbols! The Godel Code is shown in Figure 100. Compare it with the Genetic Code (Fig. 94).

There is something almost mystical in seeing the deep sharing of such an abstract structure by these two esoteric, yet fundamental, advances in knowledge achieved in our century. This Central Dogmap is by no means a rigorous proof of identity of the two theories; but it clearly shows a profound kinship, which is worth deeper exploration.

Strange Loops in the Central Dogmap

One of the more interesting similarities between the two sides of the map is the way in which "loops" of arbitrary complexity arise on the top level of both: on the left, proteins which act on proteins which act on proteins and so on, ad infinitum; and on the right, statements about statements about statements of meta-TNT and so on, ad infinitum. These are like heterarchies, which we discussed in Chapter V, where a sufficiently complex substratum allows high-level Strange Loops to occur and to cycle around, totally sealed off from lower levels. We will explore this idea in greater detail in Chapter XX.

Incidentally, you may be wondering about this question: "What, according to the Central Dogmap, is Godel's Incompleteness Theorem itself mapped onto?" This is a good question to think about before reading ahead.

The Central Dogmap and the Contracrostipuntus

It turns out that the central dogmap is quite similar to the mapping that was laid out in Chapter IV between the Contracrostipunctus and Godel’s Theorem. One can therefore drew parallels between all three systems.

The Gödel Code.

	6	2	1	3	
6	0	∀	∨	:	6
	0	∀	∨	:	2
	a	∀	*punc.*	*punc.*	1
	a	∀	*punc.*	⊃	3
2	a	~	<	·	6
	a	~	<	·	2
	a	~	>	·	1
	a	~	>	·	3
1	∧	S	+	∀	6
	∧	S	+	∀	2
	∧	S	=	·	1
	'	S	=	·	3
3	(	)	[	∃	6
	(	)	[	∃	2
	(	)	]	∃	1
	(	)	]	∃	3

FIGURE 100. The Godel Code. Under this Godel-numbering scheme, each T V7- s'mbol gets one or more codons. The small ovals show how this table subsumes the earlier Godelnumhering table of Chapter IX.

(1) formal systems and strings
(2) cells and strands of DNA
(3) record players and records

In the following chart, the mapping between systems 2 and 3 is explained carefully

Contracrostipunctus		Molecular Biology
phonograph	⇔	cell
"Perfect" phonograph	⇔	"Perfect" cell
record	⇔	strand of DNA
record playable by a given phonograph	⇔	strand of DNA reproducible by a given cell
record unplayable by that phonograph	⇔	strand of DNA unreproducible by that cell
process of converting record grooves into sounds	⇔	process of transcription of DNA onto mRNA
sounds produced by record player	⇔	strands of messenger RNA
translation of sounds into vibrations of phonograph	⇔	translation of mRNA into proteins
mapping from external sounds onto vibrations of phonograph	⇔	Genetic Code (mapping from mRNA triplets onto amino acids)
breaking of phonograph	⇔	destruction of the cell
Title of song specially tailored for Record Player X: "I Cannot Be Played on Record Player X"	⇔	High-level interpretation of DNA strand specially tailored for Cell X: "I Cannot Be Replicated by Cell X"
"Imperfect" Record Player	⇔	Cell for which there exists at least one DNA strand which it cannot reproduce
"Tödel's Theorem": "There always exists an unplayable record, given a particular phonograph."	⇔	Immunity Theorem: "There always exists an unreproducible DNA strand, given a particular cell."

The analogue of Godel's Theorem is seen to be a peculiar fact, probably little useful to molecular biologists (to whom it is likely quite obvious):

> It is always possible to design a strand of DNA which, if injected into a cell, would, upon being transcribed, cause such proteins to be manufactured as would destroy the cell (or the DNA), and thus result in the non-reproduction of that DNA

This conjures tip a somewhat droll scenario, at least if taken in light of evolution: an invading species of virus enters a cell by some surreptitious

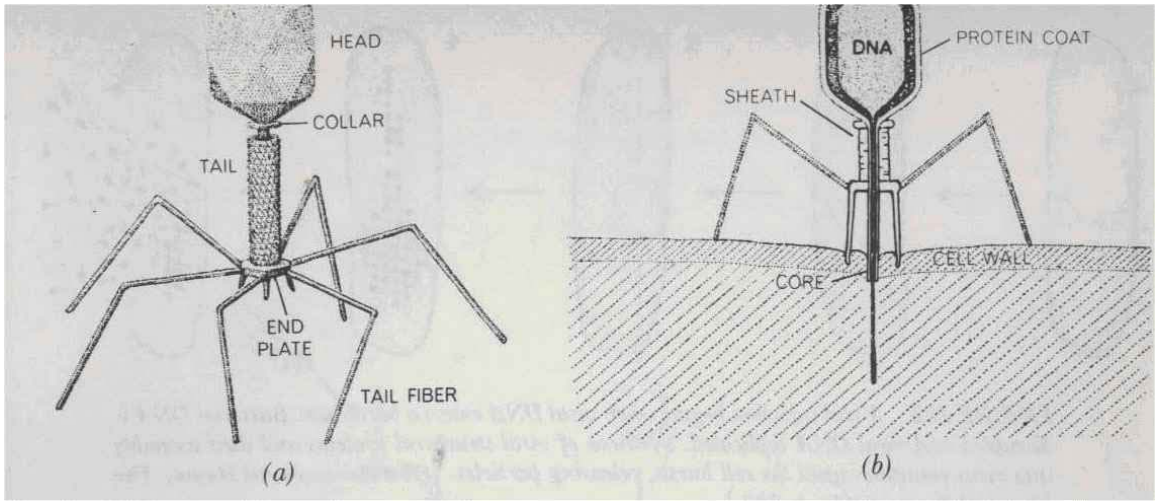


FIGURE 101. The T4 bacterial virus is an assembly of protein components (a). The "head" is a protein membrane, shaped like a kind of prolate irosahedron with thirty facets and filled with DNA. It is attached by a neck to a tail consisting of a hollow core surrounded by a contractile sheathh and based on a spiked end plate to which six fibers are attached. The spikes and fibers affix the virus to a bacterial cell wall (h). The sheath contracts, driving the core through the wall, and viral DNA enters the cell. [From Hanawalt and Haynes, The Chemical Basis of Life, p. 230.1

means, and then carefully ensures the manufacture of proteins which will have the effect of destroying the virus itself! It is a sort of suicide-or Epimenides sentence, if you will-on the molecular level. Obviously it would not prove advantageous from the point of view of survival of the species. However, it demonstrates the spirit, if not the letter, of the mechanisms of protection and subversion which cells and their invaders have developed.

E. Coli vs. T4

Let us consider the biologists' favorite cell, that of the bacterium Escherichia coli (no relation to M. C. Escher), and one of their favorite invaders of that cell: the sinister and eerie T4 phage, pictures of which you can see in Figure 101. (Incidentally, the words "phage" and "virus" are synonymous and mean "attacker of bacterial cells".) The weird tidbit looks like a little like a cross between a LEM (Lunar Excursion Module) and a mosquito-and it is much more sinister than the latter. It has a "head" wherein is stored all its "knowledge"-namely its DNA; and it has six "legs" wherewith to fasten itself to the cell it has chosen to invade; and it has a "stinging tube" (more properly called its "tail") like a mosquito. The major difference is that unlike a mosquito, which uses its stinger for sucking blood, the T4 phage uses its stinger for injecting its hereditary substance into the cell against the will of its victim. Thus the phage commits "rape" on a tiny scale.

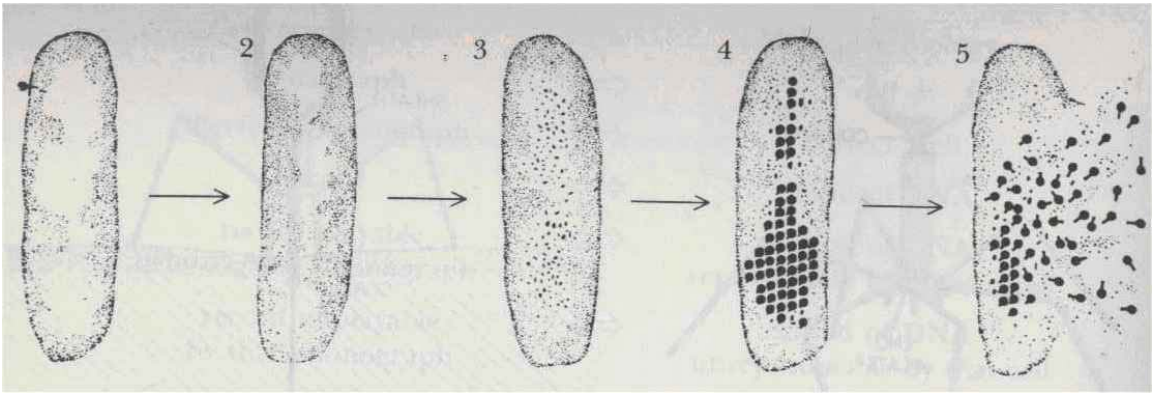


FIGURE 102. Viral infection begins when viral DNA enters a bacterium. Bacterial DNA is disrupted and viral DNA replicated. Synthesis of viral structural proteins and their assembly into virus continues until the cell bursts, releasing particles. [From Hanawalt and Haynes, The Chemical Basis of Life, p. 230.]

A Molecular Trojan Horse

What actually happens when the viral DNA enters a cell? The virus "hopes", to speak anthropomorphically, that its DNA will get exactly the same treatment as the DNA of the host cell. This would mean getting transcribed and translated, thus allowing it to direct the synthesis of its own special proteins, alien to the host cell, which will then begin to do their thing. This amounts to secretly transporting alien proteins "in code" (viz., the Genetic Code) into the cell, and then "decoding" (viz., producing) them. In a way this resembles the story of the Trojan horse, according to which hundreds of soldiers were sneaked into Troy inside a harmless seeming giant wooden horse; but once inside the city, they broke loose and captured it. The alien proteins, once they have been "decoded" (synthesized) from their carrier DNA, now jump into action. The sequence of actions directed by the T4 phage has been carefully studied, and is more or less as follows (see also Figs. 102 and 103):

Time elapsed	Action taking place
0 min.	Injection of viral DNA.
1 min.	Breakdown of host DNA. Cessation of production of native proteins and initiation of production of alien (T4) proteins. Among the earliest produced proteins are those which direct the replication of the alien (T4) DNA.
5 min.	Replication of viral DNA begins.
8 min.	Initiation of production of structural proteins which will form the "bodies" of new phages.

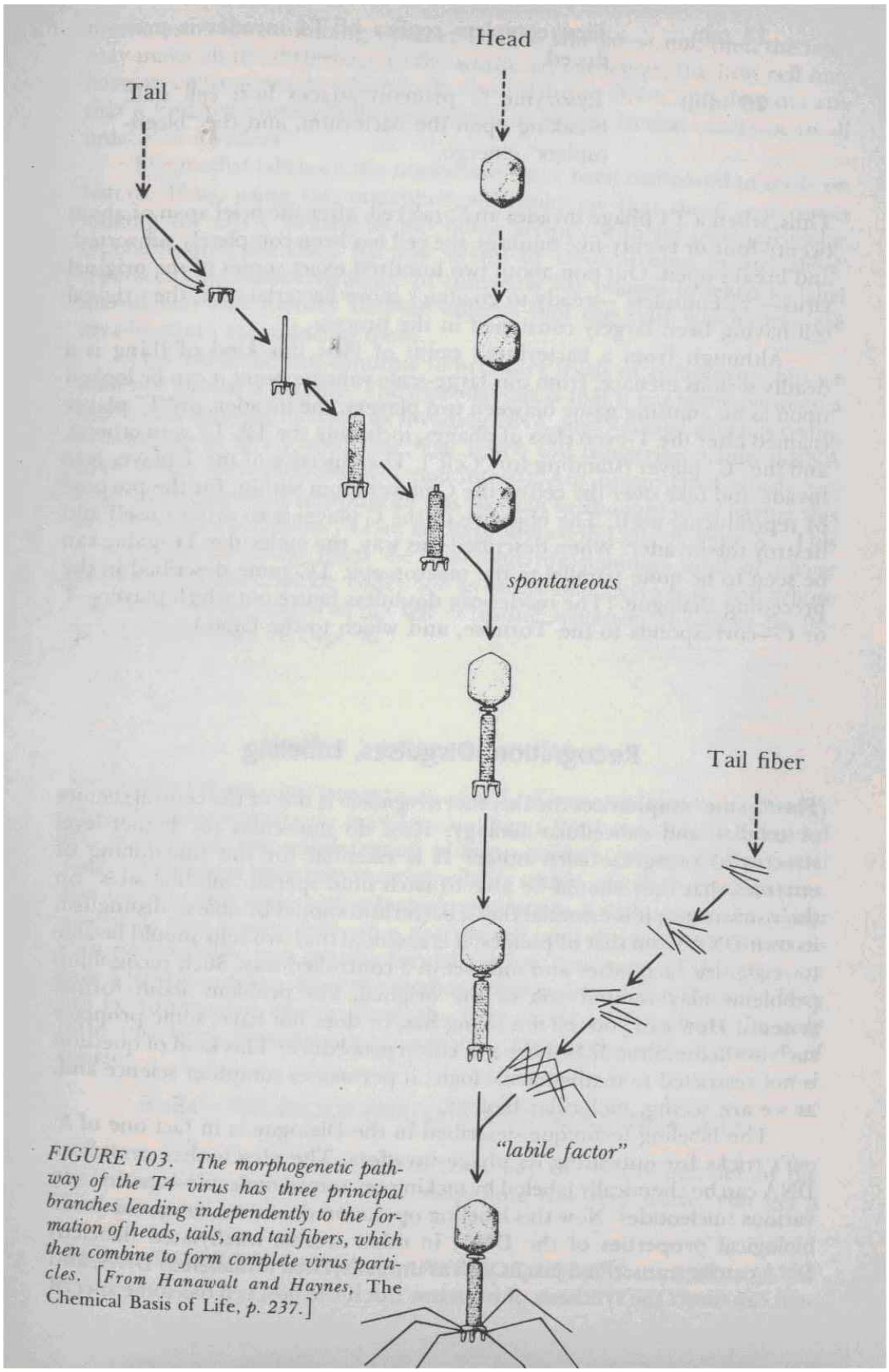


FIGURE 103. The morphogenetic pathway of the T4 virus has three principal branches leading independently to the formation of heads, tails, and tail fibers, which then combine to form complete virus particles. [*From Hanawalt and Haynes,* The Chemical Basis of Life, *p. 237.*]

13 min. First complete replica of T4 invader is produced.
25 min. Lysozyme (a protein) attacks host cell wall. breaking open the bacterium, and the "bicentuplets" emerge.

Thus, when a T4 phage invades an E. coli cell, after the brief span of about twenty-four or twenty-five minutes, the cell has been completely subverted, and breaks open. Out pop about two hundred exact copies of the original virus-"bicentuplets"-ready to go attack more bacterial cells, the original cell having been largely consumed in the process.

Although from a bacterium's point of view this kind of thing is a deadly serious menace, from our large-scale vantage point it can be looked upon as an amusing game between two players: the invader, or "T" player (named after the T-even class of phages, including the T2, T4, and others), and the "C" player (standing for "Cell"). The objective of the T player is to invade and take over the cell of the C player from within, for the purpose of reproducing itself. The objective of the C player is to protect itself and destroy the invader. When described this way, the molecular TC-game can be seen to be quite parallel to the macroscopic TC-game described in the preceding Dialogue. (The reader can doubtless figure out which player-T or C-corresponds to the Tortoise, and which to the Crab.)

Recognition, Disguises, Labeling

.

This "game" emphasizes the fact that recognition is one of the central themes of cellular and subcellular biology. How do molecules (or higher-level structures) recognize each other? It is essential for the functioning of enzymes that they should be able to latch onto special "binding sites" on their substrates; it is essential that a bacterium should be able to distinguish its own DNA from that of phages; it is essential that two cells should be able to recognize each other and interact in a controlled way. Such recognition problems may remind you of the original, key problem about formal systems: How can you tell if a string has, or does not have, some property such as theoremhood? Is there a decision procedure? This kind of question is not restricted to mathematical logic: it permeates computer science and, as we are seeing, molecular biology.

The labeling technique described in the Dialogue is in fact one of E. colt's tricks for outwitting its phage invaders. The idea is that strands of DNA can be chemically labeled by tacking on a small molecule-methyl-to various nucleotides. Now this labeling operation does not change the usual biological properties of the DNA; in other words, methylated (labeled) DNA can be transcribed just as well as unmethylated (unlabeled) DNA, and so it can direct the synthesis of proteins. But if the host cell has some special

mechanisms for examining whether DNA is labeled or not, then the label may make all the difference in the world. In particular, the host cell may have an enzyme system which looks for unlabeled DNA, and destroys any that it finds by unmercifully chopping it to pieces. In that case, woe to all unlabeled invaders!

The methyl labels on the nucleotides have been compared to serifs on letters. Thus, using this metaphor, we could say that the E. colt cell is looking for DNA written in its "home script", with its own particular typeface-and will chop up any strand of DNA written in an "alien" typeface. One counterstrategy, of course, is for phages to learn to label themselves, and thereby become able to fool the cells which they are invading into reproducing them.

This TC-battle can continue to arbitrary levels of complexity, but we shall not pursue it further. The essential fact is that it is a battle between a host which is trying to reject all invading DNA, and a phage which is trying to infiltrate its DNA into some host which will transcribe it into mRNA (after which its reproduction is guaranteed). Any phage DNA which succeeds in getting itself reproduced this way can be thought of as having this high-level interpretation: "I Can Be Reproduced in'Cells of Type X". This is to be distinguished from the evolutionarily pointless kind of phage mentioned earlier, which codes for proteins that destroy it, and whose high-level interpretation is the self-defeating sentence: "I Cannot Be Reproduced in Cells of Type X".

Henkin Sentences and Viruses

Now both of these contrasting types of self-reference in molecular biology have their counterparts in mathematical logic. We have already discussed the analogue of the self-defeating phages-namely, strings of the G6del type, which assert their own unproducibility within specific formal sstems. But one can also make a counterpart sentence to a real phage: the' phage asserts its own producibility in a specific cell, and the sentence asserts its own producibility in a specific formal system. Sentences of this type are called Henkin sentences, after the mathematical logician Leon Henkin. They can be constructed exactly along the lines of Godel sentences, the only difference being the omission of a negation. One begins with an "uncle", of course:

3a:3a':<TNT-PROOF-PAIR{a,a'}-°ARITHMOQUINE{a",a'}>

and then proceeds by the standard trick. Say the Godel number of the above "uncle" is h. Now by arithmoquining this very uncle, you get a Henkin sentence:

3a:3a':<TNT-PROOF-PAIR{a,a'} ^ ARITHMOQUINE{SSS ... SSSO/a",a'}> M

(By the way, can you spot how this sentence differs from -G?) The reason I show it explicitly is to point out that a Henkin sentence does not give a full recipe for its own derivation; it just asserts that there exists one. You might well wonder whether its claim is justified. Do Henkin sentences indeed possess derivations? Are they, as they claim, theorems? It is useful to recall that one need not believe a politician who says. "I am honest"-he may be honest, and yet he may not be. Are Henkin sentences any more trustworthy than politicians? Or do Henkin sentences, like politicians, lie in cast-iron sinks?

It turns out that these Henkin sentences are invariably truth tellers. Why this is so is not obvious; but we will accept this curious fact without proof.

Implicit vs. Explicit Henkin Sentences

I mentioned that a Henkin sentence tells nothing about its own derivation; it just asserts that one exists. Now it is possible to invent a variation on the theme of Henkin sentences-namely sentences which explicitly describe their own derivations. Such a sentence's high-level interpretation would not be "Some Sequence of Strings Exists Which is a Derivation of Me", but rather, "The Herein-described Sequence of Strings Is a Derivation of Me". Let us call the first type of sentence an implicit Henkin sentence. The new sentences will be called explicit Henkin sentences, since they explicitly describe their own derivations. Note that, unlike their implicit brethren, explicit Henkin sentences need not be theorems. In fact, it is quite easy to write a string which asserts that its own derivation consists of the single string 0=0-a false statement, since 0=0 is not a derivation of anything. However, it is also possible to write an explicit Henkin sentence which is a theorem-that is, a sentence which in fact gives a recipe for its own derivation.

Henkin Sentences and Self-Assembly

The reason I bring up this distinction between explicit and implicit Henkin sentences is that it corresponds very nicely to a significant distinction between types of virus. There are certain viruses, such as the so-called "tobacco mosaic virus", which are called self-assembling viruses; and then there are others, such as our favorite T-evens, which are non-self-assembling. Now what is this distinction? It is a direct analogue to the distinction between implicit and explicit Henkin sentences.

The DNA of a self-assembling virus codes only for the parts of a new virus, but not for any enzymes. Once the parts are produced, the sneaky virus relies upon them to link up to each other without help from any enzymes. Such a process depends on chemical affinities which the parts have for each other, when swimming in the rich chemical brew of a cell. Not only viruses, but also some organelles-such as ribosomes-assemble

themselves. Somtiems enzymes may be needed – but in such cases, they are recruited from the host cell, and enslaved. This is what is meant by self-assembly.

By contrast, the DNA of more complex viruses, such as the T-evens, codes not only for the parts, but in addition for various enzymes which play special roles in the assembly of the parts into wholes. Since the assembly process is not spontaneous but requires "machines", such viruses are not considered to be self-assembling. The essence of the distinction, then, between self-assembling units and non-self-assembling units is that the former get away with self-reproduction without telling the cell anything about their construction, while the latter need to give instructions as to how to assemble themselves.

Now the parallel to Henkin sentences, implicit and explicit, ought to be quite clear. Implicit Henkin sentences are self-proving but do not tell anything at all about their proofs-they are analogous to self-assembling viruses; explicit Henkin sentences direct the construction of their own proofs-they are analogous to more complex viruses which direct their host cells in putting copies of themselves together.

The concept of self-assembling biological structures as complex as viruses raises the possibility of complex self-assembling machines as well. Imagine a set of parts which, when placed in the proper supporting environment, spontaneously group themselves in such a way as to form a complex machine. It seems unlikely, yet this is quite an accurate way to describe the process of the tobacco mosaic virus' method of selfreproduction via self-assembly. The information for the total conformation of the organism (or machine) is spread about in its parts; it is not concentrated in some single place.

Now this concept can lead in some strange directions, as was shown in the Edifying Thoughts of a Tobacco Smoker. There, we saw how the Crab used the idea that information for self-assembly can be distributed around, instead of being concentrated in a single place. His hope was that this would prevent his new phonographs from succumbing to the Tortoise's phonograph-crashing method. Unfortunately, just as with the most sophisticated axiom schemata, once the system is all built and packaged into a box, its well-definedness renders it vulnerable to a sufficiently clever "Godelizer"; and that was the sad tale related by the Crab. Despite its apparent absurdity, the fantastic scenario of that Dialogue is not so far from reality, in the strange, surreal world of the cell.

Two Outstanding Problems: Differentiation and Morphogenesis

Now self-assembly may be the trick whereby certain subunits of cells are constructed, and certain viruses-but what of the most complex macroscopic structures, such as the body of an elephant or a spider, or the shape of a Venus's-Hyt-ap? How are homing instincts built into the brain of" a

bird, or hunting instincts into the brain of a dog% In short, how is it that merely by dictating which proteins are to be produced in cells, DNA exercises such spectacularly precise control over the exact structure and function of macroscopic living objects? There are two major distinct problems here. One is that of cellular differentiation: how do different cells, sharing exactly the same DNA, perform different roles-such as a kidney cell, a bone marrow cell, and a brain cell? The other is that of morphogenesis ("birth of form"): how does intercellular communication on a local level give rise to large-scale, global structures and organizations-such as the various organs of the body, the shape of the face, the suborgans of the brain, and so on? Although both cellular differentiation and rnorphogenesis are poorly understood at present. the trick appears to reside in exquisitely fine-tuned feedback and "feedforward" mechanisms within cells and between cells, which tell a cell when to "turn on" and when to "turn off" production of various proteins.

Feedback and Feedforward

Feedback takes place when there is too much or too little of some desired substance in the cell: then the cell must somehow regulate the production line which is assembling that substance. Feedforward also involves the regulation of" an assembly line, but not according to the amount of end product present: rather, according to the amount of some precursor of the end product of that assembly line. There are two major devices for achieving negative feedforward or feedback. One way is to prevent the relevant enzymes from being able to perform-that is, to "clog up" their active sites. This is called inhibition. The other way is to prevent the relevant enzymes from ever being manufactured! This is called repression. Conceptually, inhibition is simple: you just block up the active site of the first enzyme in the assembly line, and the whole process of synthesis gets stopped dead.

Repressors and Inducers

Repression is trickier. How does a cell stop a gene from being expressed? The answer is, it prevents it from ever getting transcribed. This means that it has to prevent RNA polymerase from doing its job. This can be accomplished by placing a huge obstacle in its path, along the DNA. precisely in front of that gene which the cell wants not to get transcribed. Such obstacles do exist, and are called repressors. They are themselves proteins, and they bind to special obstacle-holding sites on the DNA, called (I am not sure why) operators. An operator therefore is a site of control for the gene (or genes) which immediately follow it: those genes are called its operon. Because a series of enzymes often act in concert in carrying out a long chemical transformation, they are often coded for in sequence; and this is why operons often contain several genes, rather than just one. The effect of the successful repression of an operon is that a whole series of genes is

prevented from being transcribed, which means that a whole set of related enzymes remains unsynthesized.

What about positive feedback and feedforward? Here again, there are two options: (1) unclog the clogged enzymes, or (2) stop the repression of the relevant operon. (Notice how nature seems to love double-negations! Probably there is some very deep reason for this.) The mechanism by which repression is repressed involves a class of molecules called inducers. The role of an inducer is simple: it combines with a repressor protein before the latter has had a chance to bind to an operator on a DNA molecule; the resulting "repressor-inducer complex" is incapable of binding to an operator, and this leaves the door open for the associated operon to be transcribed into mRNA and subsequently translated into protein. Often the end product or some precursor of the end product can act as an inducer.

Feedback and Strange Loops Compared

Incidentally, this is a good time to distinguish between simple kinds of feedback, as in the processes of inhibition and repression, and the looping-hack between different informational levels, shown in the Central Dogrnap. Both are "feedback" in some sense; but the latter is much deeper than the former. When an amino acid, such as tryptophan or isoleucine, acts as feedback (in the form of an inducer) by binding to its repressor so that more of it gets made, it is not telling how to construct itself; it is just telling enzymes to make more of it. This could be compared to a radio's volume, which, when fed through a listener's ears, may cause itself to be turned down or up. This is another thing entirely from the case in which the broadcast itself tells you explicitly to turn your radio on or off, or to tune to another wavelength-or even how to build another radio! The latter is much more like the looping-back between informational levels, for here, information inside the radio signal gets "decoded" and translated into mental structures. The radio signal is composed of symbolic constituents whose symbolic meaning matters-a case of use, rather than mention. On the other hand, when the sound is just too loud, the symbols are not conveying meaning: they are merely being perceived as loud sounds, and might as well be devoid of meaning-a case of mention, rather than use. This case more resembles the feedback loops by which proteins regulate their own rates of synthesis.

It has been theorized that the difference between two neighboring cells which share the exact same genotype and yet have different functions is that different segments of their genome have been repressed, and therefore they have different working sets of proteins. A hypopothesis like this could account for the phenomenal differences between cells in different organs of the body of a human being.

Two Simple Examples of Differentiation

The process by which one initial cell replicates over and over, giving rise to a myriad of differentiated cells with specialized functions, can be likened to the spread of a chain letter from person to person, in which each new participant is asked to propagate the message faithfully, but also to add some extra personal touch. Eventually, there will be letters which are tremendously different from each other.

Another illustration of the ideas of differentiation is provided by this extremely simple computer analogue of a differentiating self-rep. Consider a very short program which is controlled by an up-down switch, and which has an internal parameter N-a natural number. This program can run in two modes-the up-mode, and the down-mode. When it runs in the upmode, it self-replicates into an adjacent part of the computer's memoryexcept it makes the internal parameter N of its "daughter" one greater than in itself. When it runs in the down-mode, it does not self-rep, but instead calculates the number

$$(-1)'/(2N + 1)$$

and adds it to a running total.

Well, suppose that at the beginning, there is one copy of the program in memory, N = 0, and the mode is up. Then the program will copy itself next door in memory, with N = 1. Repeating the process, the new program will self-rep next door to itself, with a copy having N = 2. And over and over again ... What happens is that a very large program is growing inside memory. When memory is full, the process quits. Now all of memory can be looked upon as being filled with one big program, composed of many similar, but differentiated, modules-or "cells". Now suppose we switch the mode to down, and run this big program. What happens? The first "cell" runs, and calculates 1/1. The second "cell" runs, calculating -1/3, and adding it to the previous result. The third "cell" runs, calculating + 1/5 and adding it on. .. The end result is that the whole "organism"-the big program-calculates the sum

$$l -1/3 +1/5 -117 +1/9 -1/11 +1/13 -1/15 + .. .$$

to a large number of terms (as many terms as "cells" can fit inside memory). And since this series converges (albeit slowlv) to 7r/4, we have a "phenotype" whose function is to calculate the value of a famous mathematical constant.

Level Mixing in the Cell

I hope that the descriptions of processes such as labeling, self-assembly, differentiation, morphogenesis, as well as transcription and translation, have helped to convey some notion of the immensely complex system which is a cell-an information-processing system with some strikingly

novel features. We have seen, in the Central Dogmap, that although we can try to draw a clear line between program and data, the distinction is somewhat arbitrary. Carrying this line of thought further, we find that not only are program and data intricately woven together, but also the interpreter of programs, the physical processor, and even the language are included in this intimate fusion. Therefore, although it is possible (to some extent) to draw boundaries and separate out the levels, it is just as important-and ust as fascinating-to recognize the level-crossings and mixings. Illustrative of this is the amazing fact that in biological systems, all the various features necessary for self-rep (viz., language, program, data, interpreter, and processor) cooperate to such a degree that all of them are replicated simultaneously-which shows how much deeper is biological self-rep'ing than anything yet devised along those lines by humans. For instance, the self-rep program exhibited at the beginning of this Chapter takes for granted the pre-existence of three external aspects: a language, an interpreter, and a processor, and does not replicate those.

Let us try to summarize various ways in which the subunits of a cell can be classified in computer science terms. First, let us take DNA. Since DNA contains all the information for construction of proteins., which are the active agents of the cell, DNA can be viewed as a program written in a higher-level language, which is subsequently translated (or interpreted) into the "machine language" of the cell (proteins). On the other hand, DNA is itself a passive molecule which undergoes manipulation at the hands of various kinds of enzymes; in this sense, a DNA molecule is exactly like a long piece of data, as well. Thirdly, DNA contains the templates off of which the tRNA "flashcards" are rubbed, which means that DNA also contains the definition of its own higher-level language.

Let us move on to proteins. Proteins are active molecules, and carry out all the functions of the cell; therefore it is quite appropriate to think of them as programs in the "machine language" of the cell (the cell itself being the processor). On the other hand, since proteins are hardware and most programs are software, perhaps it is better to think of the proteins as processors. Thirdly, proteins are often acted upon by other proteins, which means that proteins are often data. Finally, one can view proteins as interpreters; this involves viewing DNA as a collection of' high-level language programs, in which case enzymes are merely carrying out the programs written in the DNA code, which is to say, the proteins are acting as interpreters.

Then there are ribosomes and tRNA molecules. They mediate the translation from DNA to proteins, which can be compared to the translation of a program from a high-level language to a machine language; in other words, the ribosomes are functioning as interpreters and the tRNA molecules provide the definition of the higher-level language. But an alternative view of translation has it that the ribosomes are processors, while the tRNA's are interpreters.

We have barely scratched the surface in this analysis of interrelations between all these biomolecules. What we have seen is that nature feels quite

comfortable in mixing levels which we tend to see as quite distinct. Actually, in computer science there is already a visible tendency to nix all these seemingly distinct aspects of an information-processing system. This is particularly so in Artificial Intelligence research, which is usually at the forefront of computer language design.

The Origin of Life

A natural and fundamental question to ask, on learning of these incredibly intricately interlocking pieces of software and hardware is: "How did they ever get started in the first place?" It is truly a baffling thing. One has to imagine some sort of a bootstrap process occurring, somewhat like that which is used in the development of new computer languages-but a bootstrap from simple molecules to entire cells is almost beyond one's power to imagine. There are various theories on the origin of life. They all run aground on this most central of all central questions: "How did the Genetic Code, along with the mechanisms for its translation (ribosomes and tRNA molecules), originate" For the moment, we will have to content ourselves with a sense of wonder and awe, rather than with an answer. And perhaps experiencing that sense of wonder and awe is more satisfying than having an answer-at least for a while.

The Magnificrab, Indeed,

It is spring, and the Tortoise and Achilles are taking a Sunday promenade in the woods together. They have decided to climb a hill at the top of which, it is said, there is a wonderful teahouse, with all sorts of delicious pastries.

Achilles: Man oh man! If a crab

Tortoise: If a crab??

Achilles: I was about to say, if a crab ever were intelligent, then surely it would be our mutual friend the Crab. Why, he must be at least two times as smart as any crab alive. Or maybe even three times as smart as any crab alive. Or perhaps

Tortoise: My soul! How you magnify the Crab!

Achilles: Well, I just happen to be an admirer of his ...

Tortoise: No need to apologize. I admire him, too. Speaking of Crab admirers, did I tell you about the curious fan letter which the Crab received not too long ago?

Achilles: I don't believe so. Who sent it?

Tortoise: It bore a postmark from India, and was from someone neither of us had ever heard of before-a Mr. Najunamar, I believe.

Achilles: I wonder why someone who never knew Mr. Crab would send him a letter-or for that matter, how they would get his address. Tortoise: Apparently whoever it was was under the illusion that the Crab is a mathematician. It contained numerous results, all of which were But, ho! Speak of the devil! Here comes Mr. Crab now, down the hill. Crab: Good-bye! It was nice to talk with you again. Well, I guess I had best be off. But I'm utterly stuffed-couldn't eat one more bite if I had to! I've just been up there myself-recommend it highly. Have you ever been to the teahouse at the crest of the hill? How are you, Achilles? Oh, there's Achilles. Hello, hello. Well, well, if it isn't Mr. T!

Tortoise: Hello, Mr. C. Are you headed up to the hilltop teahouse? Crab: Why, yes indeed, I am; how did you guess it? I'm quite looking forward to some of their special napoleons-scrumptious little morsels. I'm so hungry I could eat a frog. Oh, there's Achilles. How are you, Achilles?

Achilles: Could be worse, I suppose.

Crab: Wonderful! Well, don't let me interrupt your discussion. I'll just tag along.

Tortoise: Curiously enough, I was just about to describe your mysterious letter from that Indian fellow a few weeks back-but now that you're here. I'll let Achilles get the story from the Crab’s mouth.

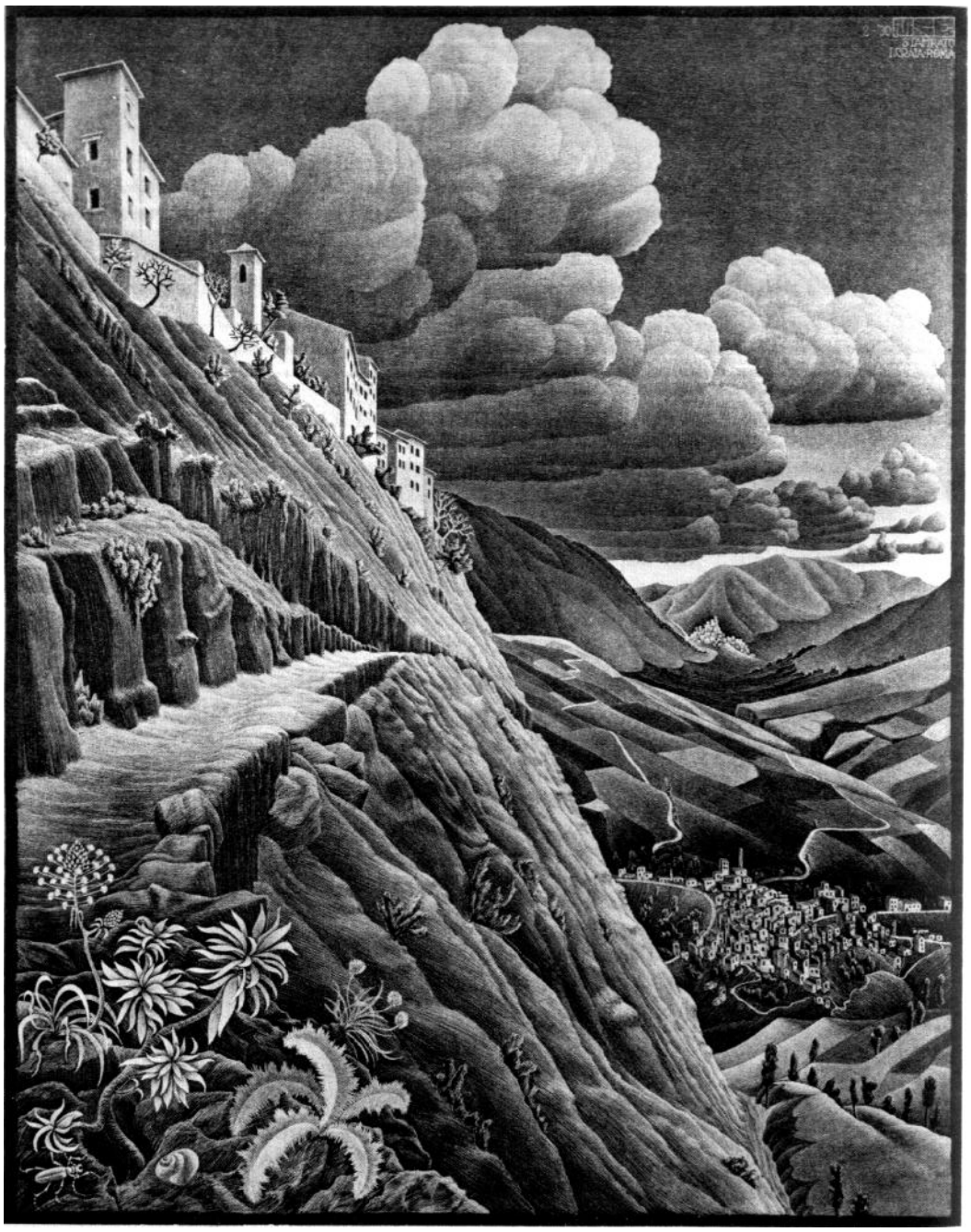


FIGURE 104. Castrovalva, by'M. C. Escher (lithograph, 1930).

Crab: Well, it was this way. This fellow Najunamar had apparently never had any formal training in mathematics, but had instead worked out some of his own methods for deriving new truths of mathematics. Some of his discoveries defeated me completely; I had never seen anything in the least like them before. For instance, he exhibited a map of India that he had managed to color using no fewer than 1729 distinct colors.

Achilles: 1729! Did you say 1729? Crab: Yes-why do you ask?

Achilles: Well, 1729 is a very interesting number, you know. Crab: Indeed. I wasn't aware of it.

Achilles: In particular, it so happens that 1729 is the number of the taxicab which I took to Mr. Tortoise's this morning!

Crab: How fascinating! Could you possibly tell me the number of the trolley car which you'll take to Mr. Tortoise's tomorrow morning?

Achilles (after a moment's thought): It's not obvious to me; however, I should think it would be very large.

Tortoise: Achilles has a wonderful intuition for these things.

Crab: Yes. Well, as I was saying, Najunamar in his letter also proved that every even prime is the sum of two odd numbers, and that there are no solutions in positive integers to the equation

$$a^n + b^n = c^n \qquad \text{for } n = 0.$$

Achilles: What? All these old classics of mathematics resolved in one fell swoop? He must be a genius of the first rank! Tortoise: But Achilles-aren't you even in the slightest skeptical?

Achilles: What? Oh, yes-skeptical. Well, of course I am. You don't think I believe that Mr. Crab got such a letter, do you? I don't fall for just anything, you know. So it must have been 5'ou, Mr. T, who received the letter!

Tortoise: Oh, no, Achilles, the part about Mr. C receiving the letter is quite true. What I meant was, aren't you skeptical about the content of the letter-its extravagant claims?

Achilles: Why should I be? Hmm ... Well, of course I am. I'm a very skeptical person, as both of you should well know by now. It's very hard to convince me of anything, no matter how true or false it is.

Tortoise: Very well put, Achilles. You certainly have a first-class awareness of your own mental workings.

Achilles: Did it ever occur to you, my friends, that these claims of Najunamar might be incorrect?

Crab: Frankly, Achilles, being rather conservative and orthodox myself, I was a bit concerned about that very point on first receiving the letter. In fact, I suspected at first that here was an out-and-out fraud. But on second thought, it occurred to me that not many types of people could manufacture such strange-sounding and complex results purely from their imagination. In fact, what it boiled down to was this question:

"Which is the more likely: a charlatan of such extraordinary ingenuity, or a mathematician of great genius?" And before long, I realized that the probabilities clearly favored the former.

Achilles: Didn't you directly checkout any of his amazing claims, however?

Crab: Why should I? The probability argument was the most convincing thing I had ever thought of; no mathematical proof would have equaled it. But Mr. T here insisted on rigor. I finally gave in to his insistence, and checked all of Najunamar's results. To my great surprise, each one of them was right. How he discovered them, I'll never know, however. He must have some amazing and inscrutable Oriental type of insight which we here in the Occident can have no inkling of. At present, that's the only theory which makes an sense to me.

Tortoise: Mr. Crab has always been a little more susceptible to mystical or fanciful explanations than I am. I have full confidence that whatever Najunamar did in his way has a complete parallel inside orthodox mathematics. There is no way of doing mathematics which is fundamentally different from what we now know, in my opinion.

Achilles: That is an interesting opinion. I suppose it has something to do with the Church-Turing Thesis and related topics.

Crab: Oh, well, let us leave these technical matters aside on such a fine day, and enjoy the quiet of the forest, the chirping of the birds, and the play of sunlight on the new leaves and buds. Ho!

Tortoise: I second the motion. After all, all generations of Tortoises have reveled in such delights of nature.

Crab: As have all generations of Crabs.

Achilles: You don't happen to have brought your flute along, by any chance, Mr. C?

Crab: Why, certainly! I take it with me everywhere. Would you like to hear a tune or two?

Achilles: It would be delightful, in this pastoral setting. Do you play from memory?

Crab: Sad to say, that is beyond my capability. I have to read my music
from a sheet. But that is no problem. I have several very pleasant pieces here in this case.

(He opens up a thin case and draws out a few pieces of paper. The topmost one has the following symbols on it:

Va:-Sa=O

He sticks the top sheet into a little holder attached to his flute, and plays. The tune is very short.)

Achilles: That was charming. (Peers over at the sheet on the flute, and a quizzical expression beclouds his face.) What is that statement of number theory doing, attached to your flute like that?

(The Crab looks at his flute, then his music, turns his head all around, and appears slightly confused.)

Crab: I don't understand. What statement of number theory?
Achilles: "Zero is not the successor of any natural number." Right there, in the holder on your flute!
Crab: That's the third Piano Postulate. There are five of them, and I've arranged them all for flute. They're obvious, but catchy.
Achilles: What's not obvious to me is how a number-theoretical statement can be played as music.
Crab: But I insist, it's 'NOT a number-theoretical statement-it's a Piano Postulate! Would you like to hear another?
Achilles: I'd be enchanted.

(The Crab places another piece of paper on his flute, and this time Achilles watches more carefully.)

Well, I watched your eyes, and they were looking at that FORMULA on the sheet. Are you sure that that is musical notation? I swear, it most amazingly resembles the notation which one might use in a formalized version of number theory.
Crab: How odd! But certainly that is music, not any kind of statement of mathematics, as far as I can tell! Of course, I am not a mathematician in any sense of the word. Would you like to hear any other tunes?
Achilles: By all means. Have you some others?
Crab: Scads.

(He takes a new sheet, and attaches it to his flute. It contains the following symbols:

3a:3b:(SSa. SSb)=SSSSSSSSSSSSSO

Achilles peers at it, while the Crab plays it.)

Isn't it lovely?
Achilles: Yes, it certainly is a tuneful little piece. But I have to say, it's looking more and more like number theory to me.
Crab: Heavens! It is just my usual music notation, nothing more. I simply don't know how you read all these extramusical connotations into a straightforward representation for sounds.
Achilles: Would you be averse to playing a piece of my own composition?
Crab: Not in the least. Have you got it with you?
Achilles: Not yet, but I have a hunch I might be able to compose some tunes all by myself.
Tortoise: I must. tell you, Achilles, that Mr. C is a harsh judge of music composed by others, so do not be disappointed if, by some chance, he is not an enthusiast for your efforts.
Achilles: That is very kind of you to forewarn me. Still, I'm willing to give it a try .

(He writes:

((SSSO . SSSO) +(SSSSO. SSSSO))=(SSSSSO. SSSSSO)

The Crab takes it, looks it over for a moment, then sets it in his music holder, and pipes.)

Crab: Why, that's quite nice, Achilles. I enjoy strange rhythms.
Achilles: What's strange about the rhythms in that piece?
Crab: Oh, naturally, to you as the composer it must seem quite bland, but to my ears, shifting from a 3/3 rhythm to 4/4 and then to 5/5 is quite exotic. If you have any other songs, I'd be glad to play them. Achilles: Thank you very much. I've never composed anything before, and I must say composing is quite different from how I had imagined it to be. Let me try my hand at another one. (*jots down a line*.)

3a:3b:(SSa - SSb) =SSSSSSSSSSSSSSO

Crab: Hmmm ... Isn't that just a copy of my earlier piece?
Achilles: Oh, no! I've added one more S. Where you had thirteen in a row, I have fourteen.
Crab: Oh, yes. Of course. (*He plays it, and looks very stern*.)
Achilles: I do hope you didn't dislike my piece!
Crab: I am afraid, Achilles, that you completely failed to grasp the subtleties of my piece, upon which yours is modeled. But how could I expect you to understand it on first hearing? One does not always understand what is at the root of beauty. It is so easy to mistake the superficial aspects of a piece for its beauty, and to imitate them, when the beauty itself is locked deep inside the music, in a way which seems always to elude analysis.
Achilles: I am afraid that you have lost me a little in your erudite commentary. I understand that my piece does not measure up to your high standards, but I do not know exactly where I went astray. Could you perhaps tell me some specific way in which you find fault with my composition?
Crab: One possible way to save your composition, Achilles, would be to insert another three S's-five would do as well-into that long group of S's near the end. That would create a subtle and unusual effect.
Achilles: I see.
Crab: But there are other ways you might choose to change your piece. Personally, I would find it most appealing to put another tilde in the front. Then there would be a nice balance between the beginning and the end. Having two tildes in a row never fails to give a gay little twist to .a piece, you know.
Achilles: How about if I take both of your suggestions, and make the following piece?

-3a:3b:(SSa.SSb)=SSSSSSSSSSSSSSSSSO

Crab (a painful grimace crossing his face): Now, Achilles, it is important to learn the following lesson: never try to put too much into any single piece. There is always a point beyond which it cannot be improved,

and further attempts to improve it will in fact destroy it. Such is the case in this example. Your idea of incorporating both of my suggestions together does not yield the desired extra amount of beauty, but on the contrary creates an imbalance which quite takes away all the charm.

Achilles: How is it that two very similar pieces, such as yours with thirteen 5's, and mine with fourteen S's, seem to you to be so different in their musical worth? Other than in that minor respect, the two are identical.

Crab: Gracious! There is a world of difference between your piece and mine. Perhaps this is a place where words fail to convey what the spirit can feel. Indeed, I would venture to say that there exists no set of rules which delineate what it is that makes a piece beautiful, nor could there ever exist such a set of rules. The sense of Beauty is the exclusive domain of Conscious Minds, minds which through the experience of living have gained a depth that transcends explanation by any mere set of rules.

Achilles: I will always remember this vivid clarification of the nature of Beauty. I suppose that something similar applies to the concept of Truth, as well?

Crab: Without doubt. Truth and Beauty are as interrelated as-as

Achilles: As interrelated as, say, mathematics and music?

Crab: Oh! You took the words right out of my mouth! How did you know that that is what I was thinking?

Tortoise: Achilles is very clever, Mr. C. Never underestimate the potency of his insight.

Achilles: Would you say that there could conceivably be any relationship between the truth or falsity of a particular statement of mathematics, and the beauty, or lack of beauty, of an associated piece of music? Or is that just a far-fetched fancy of mine, with no basis in reality?

Crab: If you are asking me, that is carrying things much too far. When I spoke of the interrelatedness of music and mathematics, I was speaking very figuratively, you know. As for a direct connection between specific pieces of music and specific statements of mathematics, however, I harbor extremely grave doubts about its possibility. I would humbly counsel you not to give too' much time to such idle speculations.

Achilles: You are no doubt right. It would be most unprofitable. Perhaps I ought to concentrate on sharpening my musical sensitivity by composing some new pieces. Would you be willing to serve as my mentor,

Mr. C?

Crab: I would be very happy to aid you in your steps towards musical understanding.

(So Achilles takes pen in hand, and, with what appears to be a great deal of concentration, writes:

AOOaV'\/--nn:b+cS(33=OAD((-d)<v(VS-+(>v

The Crab looks very startled.)

Y ou really want me to play that-that-that whatever-it-is?
Achilles: Oh, please do!

(So the Crab plays it, with evident difficulty.)

Tortoise: Bravo! Bravo! Is John Cage your favorite composer, Achilles? Achilles: Actually, he's my favorite anti-composer. Anyway, I'm glad you liked MY music.

Crab: The two of you may find it amusing to listen to such totally meaningless cacophony, but I assure you it is not at all pleasant for a sensitive composer to be subjected to such excruciating, empty dissonances and meaningless rhythms. Achilles, I thought you had a good feeling for music. Could it be that your previous pieces had merit merely by coincidence?

Achilles: Oh, please forgive me, Mr. Crab. I was trying to explore the limits of your musical notation. I wanted to learn directly what kinds of sound result when I write certain types of note sequences, and also how you evaluate pieces written in various styles.

Crab: Harrumph! I am not just an automatic music-machine, you know. Nor am I a garbage disposal for musical trash.

Achilles: I am very sorry. But I feel that I have learned a great deal by writing that small piece, and I am convinced that I can now write much better music than I ever could have if I hadn't tried that idea. And if you'll just play one more piece of mine, I have high hopes that you will feel better about my musical sensitivities.

Crab: Well, all right. Write it down and I'll give it a chance.

(*Achilles writes:*

Ya:Vb:<(a -a) =(SSO -(b > b))Da=0>

and the Crab plays.)

You were right, Achilles. You seem to have completely regained your musical acuity. This is a little gem! How did you come to compose it? I have never heard anything like it. It obeys all the rules of harmony, and yet has a certain-what shall I say?-irrational appeal to it. I can't put my finger on it, but I like it for that very reason.

Achilles: I kind of thought you might like it.

Tortoise: Have you got a name for it, Achilles? Perhaps you might call it "The Song of Pythagoras". You remember that Pythagoras and his followers were among the first to study musical sound.

Achilles: Yes, that's true. That would be a fine title.

Crab: Wasn't Pythagoras also the first to discover that the ratio of two squares can never be equal to 2? Tortoise: I believe you're right. It was considered a truly sinister discovery at the time, for never before had anyone realized that there are

numbers-such as the square root of 2-which are not ratios of integers. And thus the discovery was deeply disturbing to the Pythagoreans, who felt that it revealed an unsuspected and grotesque defect in the abstract world of numbers. But I don't know what this has to do with the price of tea in China.

Achilles: Speaking of tea, isn't that the teahouse just up there ahead of us?

Tortoise: Yes, that's it, all right. We ought to be there in a couple of minutes.

Achilles: Hmm ... That's just enough time for me to whistle for you the tune which the taxi driver this morning had on his radio. It went like this.

Crab: Hold on for a moment; I'll get some paper from my case, and jot down your tune. (*Scrounges around inside his case, and finds a blank sheet*.) Go ahead; I'm ready.

(Achilles whistles a rather long tune, and the Crab scrambles to keep up with him.)

Could you whistle the last few bars again?

Achilles: Why, certainly.

(After a couple of such repeats, the session is complete, and the Crab proudly displays his transcription:

<((SSSSSO.SSSSSO)+(SSSSSO.SSSSSO))=((SSSSSSSO.SSSSSSSO)+(SO.SO))n---
3b:<3c:(Sc+b)=((SSSSSSSO . SSSSSSSO)+(SO • SO))n3d:3d':3e:3e':
<---<d=evd=e' >n<b=((Sd • Sd)+(Sd' • Sd'))nb=((Se • Se)+(Se' • Se'))>> >>

The Crab then plays it himself.)

Tortoise: It's peculiar music, isn't it? It sounds a wee bit like music from India, to me.

Crab. Oh, I think it's too simple to be from India. But of course I know precious little about such things.

Tortoise: Well, here we are at the teahouse. Shall we sit outside here, on the verandah?

Crab: If you don't mind, I'd prefer to go inside. I've gotten perhaps enough sun for the day.

(They go inside the teahouse and are seated at a nice wooden table, and order cakes and tea. Soon a cart of scrumptious-looking pastries is wheeled up, and each of them chooses his favorite.)

Achilles: You know, Mr. C, I would love to know what you think of another piece which I have just composed in my head. Crab: Can you show it to me? Here, write it down on this napkin.

(Achilles writes:

da:3b:3c:<- 3d:3e:<(SSd•SSe)=bv(SSd•SSe)=c>n(a+a)=(b+c)>

The Crab and Tortoise study it with interest.)

Tortoise: Is it another beautiful piece, Mr. C, in your opinion?
Crab: Well, uh ... (Shifts in his chair, and looks somewhat uncomfortable.)
Achilles: What's the matter? Is it harder to decide whether this piece is beautiful than it is for other pieces?
Crab: Ahm ... No, it's not that-not at all. It's just that, well ... I really have to HEAR a piece before I can tell how much I like it.
Achilles: So go ahead and play it! I'm dying to know whether you find it beautiful or not.
Crab: Of course, I'd be extremely glad to play it for you. The only thing is
Achilles: Can't you play it for me? What's the matter? Why are you balking?
Tortoise: Don't you realize, Achilles, that for Mr. Crab to fulfill your request would be most impolite and disturbing to the clientele and employees of this fine establishment?
Crab (suddenly looking relieved): That's right. We have no right to impose our music on others.
Achilles (dejectedly): Oh, PHOOEY! And I so much wanted to know what he thinks of this piece!
Crab: Whew! That was a close call!
Achilles: What was that remark?
Crab: Oh-nothing. It's just that that waiter over there, he got knocked into by another waiter, and almost dropped a whole pot of tea into a lady's lap. A narrow escape, I must say. What do you say, Mr. Tortoise? Tortoise: Very good teas, I'd say. Wouldn't you agree, Achilles? Achilles: Oh, yes. Prime teas, in fact.
Crab: Definitely. Well, I don't know about you two, but I should perhaps be going, for I've a long steep trail back to my house, on the other side of this hill.
Achilles: You mean this is a big bluff?
Crab: You said it, Achilles.
Achilles: I see. Well, I'll have to remember that.
Crab: It has been such a jolly afternoon, Achilles, and I sincerely hope we will exchange more musical compositions another day.
Achilles: I'm looking forward to that very much, Mr. C. Well, good-bye. Tortoise: Good-bye, Mr. C.
(And the Crab heads off down his side of the hill.)
Achilles: Now there goes a brilliant fellow ... In my estimation, he's at least four times as smart as any crab alive. Or he might even be five
Tortoise: As you said in the beginning, and probably shall be saying forevermore, words without end.